

Table of Contents

[DatalogSDKDefTopic](#)

[dIDSKAbout.1.2](#)

[dIDSKAbout.1.3](#)

[dIDSKAbout.1.4](#)

[dIDSKAbout.1.5](#)

[dIDSKUsing.2.01](#)

[dIDSKUsing.2.02](#)

[dIDSKUsing.2.03](#)

[dIDSKUsing.2.04](#)

[dIDSKUsing.2.05](#)

[dIDSKUsing.2.06](#)

[dIDSKUsing.2.07](#)

[dIDSKUsing.2.08](#)

[dIDSKUsing.2.09](#)

[dIDSKUsing.2.10](#)

[dIDSKUsing.2.11](#)

[dIDSKUsing.2.12](#)

[dIDSKUsing.2.13](#)

[dIDSKUsing.2.14](#)

[dIDSKUsing.2.15](#)

[dIDSKUsing.2.16](#)

[dIDSKUsing.2.17](#)

[dIDSKUsing.2.18](#)

[dIDSKUsing.2.19](#)

[dIDSKUsing.2.20](#)

[dIDSKUsing.2.21](#)

[dIDSKUsing.2.22](#)

[dIDSKUsing.2.23](#)

[dIDSKUsing.2.24](#)

[dISDKUsing.2.25](#)[dISDKUsing.2.26](#)[dISDKUsing.2.27](#)[dISDKUsing.2.28](#)[dISDKUsing.2.29](#)[dISDKUsing.2.30](#)[dISDKUsing.2.31](#)[dISDKUsing.2.32](#)[dISDKUsing.2.33](#)[dISDKUsing.2.34](#)[dISDKUsing.2.35](#)[dISDKUsing.2.36](#)[dISDKUsing.2.37](#)[dISDKUsing.2.38](#)[dISDKUsing.2.39](#)[dISDKUsing.2.40](#)

DatalogSDKDefTopic



About the Datalog SDK

About the Datalog SDK

The documentation for the Datalog SDK has two major sections.

The first is a description of using the Datalog SDK to create data consumers and data containers, and of working with these in IG-XL. See [Using Datalog SDK](#)

The second section is the Datalog SDK reference. See the Contents pane of IG-XL Help to displays the contents of the reference.

The rest of this section is an overview of the Datalog SDK, along with a simple example:

- [IG-XL Datalogging Architecture Overview](#)
- [Elements of the Datalogging System](#)
- [The Datalog Software Development Kit \(SDK\)](#)
- [A Simple Data Consumer](#)

dIDSKAbout.1.2

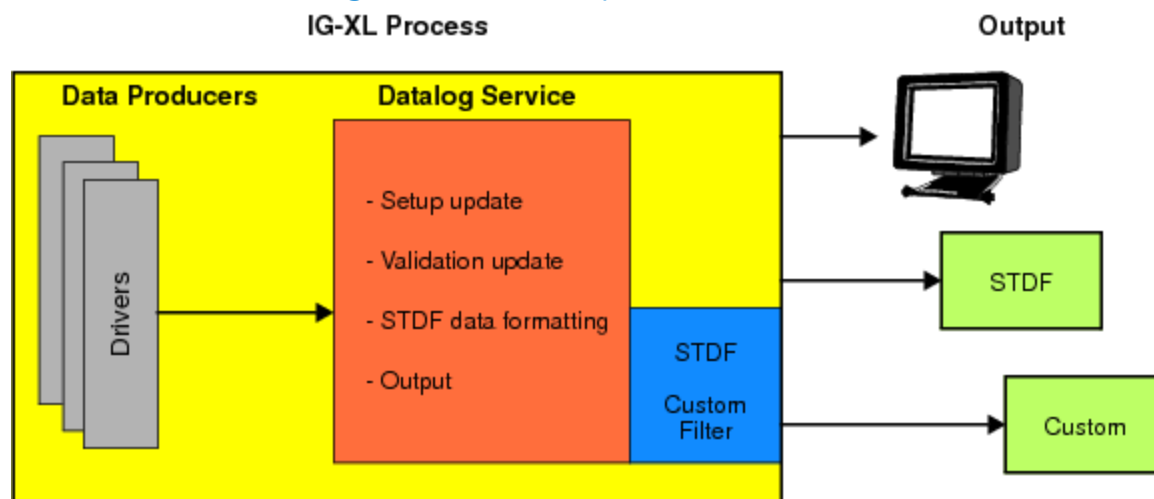


About the Datalog SDK : IG-XL Datalogging Architecture Overview

IG-XL Datalogging Architecture Overview

Data from the tester is needed for real-time monitoring and decision-making in the production environment. The data may be used to improve product quality, the cost of test, operational efficiency, and yield. These data needs can go beyond the information stored in Standard Test Data Format (STDF). The Datalog SDK provides true end-to-end customizability to capture and process data and events from the tester in real time, with limited impact on tester throughput, to build data-based and event-based solutions.

Earlier versions of the datalogging architecture immediately packaged test data into STDF data structures. To customize the data or act on it, you created an STDF custom filter, which acted on the data of interest in the STDF data stream. The STDF data formatter and custom filter ran in the same thread as IG-XL, resulting in a test time impact.



Previous IG-XL Datalog Architecture

Having access only to the STDF data stream meant that you were limited to the data that STDF could store. You could package additional data into Generic Data Records (GDR), but these are fairly rigid and limited in their structure. They also incur overhead twice: once to create the GDR record and once to process it in the STDF custom filter code.

The new IG-XL datalogging architecture provides a more scalable and customizable system. It is designed to enable multiple data consumers to manipulate the data in parallel with limited impact on test time. The following is a high-level user view of the system:



Data Producers: Any one (user, system components, tools, 3rd party apps) can post data and events to data stream through an API.

Data Consumers: Multiple consumers can act on the stream in parallel.

Current IG-XL Datalog Architecture

In the center of the system is a data stream. Data from the test program and system is posted to the data stream in a raw form and sent to registered data consumers running in parallel to the IG-XL execution thread. The data consumers act on the data asynchronously to IG-XL.

The data consumers have flexibility in what data they decide to listen to, how they process the data, and what they do with it. A data consumer can be a file generator, a GUI, a database, or a network socket. As more data consumers are added, there is a very limited impact to the test time. The only test-time overhead in this system is related to the time it takes to create data packets and post them to the data stream.

dIDSKAbout.1.3

<

>

About the Datalog SDK : Elements of the Datalogging System

Elements of the Datalogging System

The following are elements of the datalogging system.

data container	A data container is a construct that contains both data fields and methods to read and write that data. The data is stored in raw form. The system or the test program can generate data packets by populating data containers. There are a variety of built-in data containers in IG-XL, such as Event Packet, Calibration Event Packet, Generic Packet and Functional Test Result Packet. Users can also create custom data containers. All custom data containers are of type "DataContainerPackets."
data packet	A data container with data.
system event	A unique data container, with a timestamp, that the system automatically generates at some specified time and then sends to the data stream. IG-XL defines these system events: ProgramLoaded, RunStart, RunStop, StartLot, EndLot, StartWafer, EndWafer, ValidationStart, ValidationEnd, CalibrationStart,CalibrationEnd, CalibrationQuit, TDRCalibrationStart, TDRCalibrationEnd, CalibrationPowerupResetStart, CalibrationPowerupResetEnd, ChangeMode, ApplicationShutdown, and EnableWordsUpdate.
custom event	Similar to a system event, except that it is generated by the user inside the test program.
data stream	A communication layer that transports data packets from IG-XL to the datalog server process. Data packets that are generated by IG-XL

	automatically or by user action are placed on the data stream. Custom data packets can be posted to the data stream through a set of VBT commands.
data producer	Any system component that generates and posts data packets to the data stream is a data producer. Users can produce data using standard or custom data containers.
data consumer	A component that receives and processes data packets. IG-XL ships with three different data consumers: STDF, ASCII, and Data Window. Users can create custom data consumers. The data consumers can create any custom data format without the need to process STDF records. Data consumers have a high degree of flexibility in what they do with the data. They can generate files or databases, or send data over a network socket. Data consumers accept only those data packets that they are programmed to process. This lets the system add new data packet types without breaking compatibility with existing data consumers.
STDF data filters	Under the earlier datalogging architecture, this was the only mechanism to create custom output. These filters attach to the STDF data stream and can create custom output using STDF records as an input. They continue to be supported for backward compatibility.

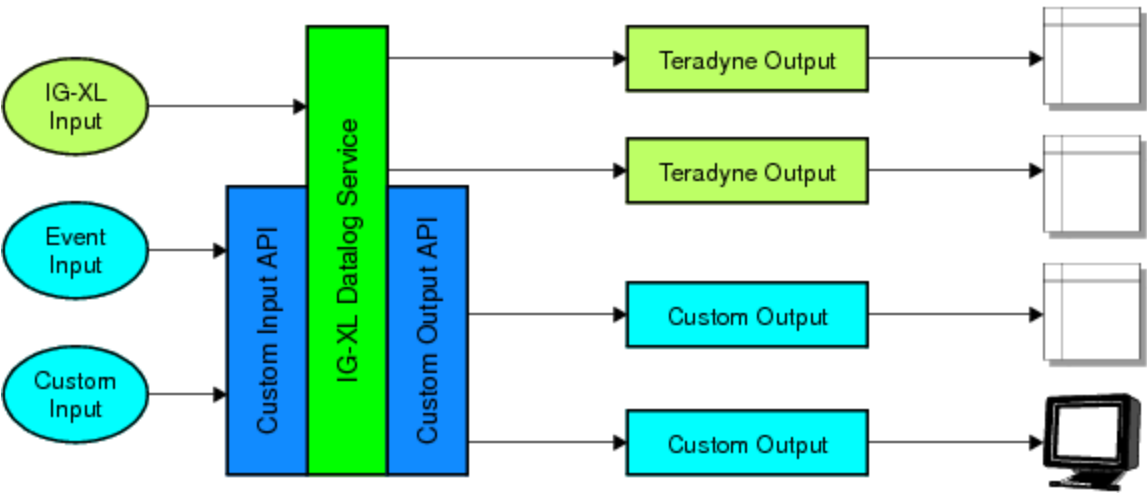
dIDSKAbout.1.4

<	>
--------------------------------------	--------------------------------------

About the Datalog SDK : The Datalog Software Development Kit (SDK)

The Datalog Software Development Kit (SDK)

The Datalog SDK consists of a set Application Programming Interfaces (APIs), user documentation, sample code, and a Microsoft Visual Studio™ project template to enable users to create their own data consumers and custom data containers. The Custom Input API allows users to create custom data containers and send them over the data stream. The Custom Output APIs allow custom data consumers to read the data on the data stream.



Datalog SDK APIs

Development of custom data packets and consumers can be done using Microsoft Visual Studio or newer managed code language such as C# or VB.net.

<	>
--------------------------------------	--------------------------------------

dIDSKAbout.1.5

<

>

About the Datalog SDK : A Simple Data Consumer

A Simple Data Consumer

A test floor manager often needs to track the number of hours a tester is used and to look for opportunities to improve the factory output. This requires the ability to log a variety of system events.

As an example of a simple data consumer, the SysEventTracker records system events in a log file. The log file can be processed to identify how time is spent on the tester.

The SysEventTracker logs the following system events to understand the different ways in which the tester time is being used:

System Events	
Time To Be Recorded	Logged System Events
Processing devices	Run Start, Run Stop
Calibration times	CalibrationStart, CalibrationEnd
Validation times	ValidationStart, ValidationEnd
TDR calibration	TDRCalibrationStart, TDRCalibrationEnd
Idle tester	ApplicationShutdown, ProgramLoaded

A mechanism is needed to pass the file path and name for the log to the SysEventTracker. To do this, a custom data container called EventFileNameContainer is created.

Creating the Custom Data Container

To set up a data container project in Microsoft Visual Studio, follow the instructions in [Setting Up a DataContainer](#). Once the project is set up, create a data container interface and class. Basically, this consists of data members in the data container and read/write methods. The custom data

container, `EventFileNameContainer`, is simple and contains two members: a string to hold the name of the event file log, and a string to hold the name of the datalog stream that it belongs to.

Creating the Data Consumer

To set up a data consumer project in Microsoft Visual Studio, follow the instructions in the [Setting Up a Custom DataConsumer](#). Once the project is set up, there are three steps to create a data consumer, as described in the following paragraphs.

Step 1: Building the Data Packet Dictionary

A dictionary is a look-up data structure. In this case, the data consumer is declaring to the system what data packets it cares about and what functions it will use to handle those data packets. The `SysEventTracker` is interested in processing only System Events containers and the custom `EventFileNameContainer`:

```
/// <summary>
/// Builds a list of methods to be called when specified DataPackets
/// are received by the Datalog system.
/// </summary>
/// <param name="outDictionary">A Dictionary, specific to this type of
/// DataConsumer, mapping DataPacket type to methods to be called when
/// that DataPacket is received.</param>
protected override void BuildDataPacketDictionary(Teradyne.Igxl.Utilities.ISearchCollection<Type,
ProcessPacketHandler> outDictionary)
{
    //System Events Packet
    outDictionary.Add(typeof(ValicationEndEventPacket), HandleValidationEndPacket);
    outDictionary.Add(typeof(TDRCalibrationDoneEventPacket), HandleTDRPacket);
    outDictionary.Add(typeof(CalibrationDoneEventPacket), HandleCalDonePacket);
    outDictionary.Add(typeof(SystemEventPacket), HandleSystemPacket);
    //Data container packets. We need this because our custom data container
    // is of this type.
    outDictionary.Add(typeof(DataContainerPacket), HandleDataContainerPacket);
}
```

Step 2: Handling Data Packets

Data packets are sent from the datalogging system to the data consumers. Data consumers can query for incoming data packets and process their properties and methods. The tester utilization tool will log the timestamp for each event. Here is an example for the system data packet handler code:

```
protected virtual void HandleSystemPacket(DataPacket inPacket)
{
    try
```

```

{
    SystemEventPacket syspacket = (SystemEventPacket)inPacket;
    if (syspacket != null)
    {
        if (m_EventsFile != null)
        {
            switch (syspacket.EventID)
            {
                case SystemEventType.ProgramLoaded;
                    m_EventsFile.WriteLine("ProgramLoaded " + new
DateTime(syspacket.TimeStamp).ToString());
                    break;

                case SystemEventType.ValidationStart;
                    m_EventsFile.WriteLine("ValidationStart " + new DateTime(syspacket.TimeStamp).ToString());
                    break;

                case SystemEventType.RunStart;
                    m_EventsFile.WriteLine("RunStart " + new DateTime(syspacket.TimeStamp).ToString());
                    break;

                case SystemEventType.RunStop;
                    m_EventsFile.WriteLine("RunStop " + new DateTime(syspacket.TimeStamp).ToString());
                    break;

                case SystemEventType.TDRCalibrationStart;
                    ...

```

Step 3: Final Cleanup and Processing Code

By implementing the `FinalizeCleanupAndProcessing` method in the data consumer, we can make sure to release any used resources. IG-XL calls this method on the data consumer prior to shutting down:

```

protected override void FinalizeCleanupAndProcessing()
{
    if (m_EventsFile != null)
    {
        m_EventsFile.Close();
    }
}

```

```
base.FinalizeCleanupAndProcessing();
}
```

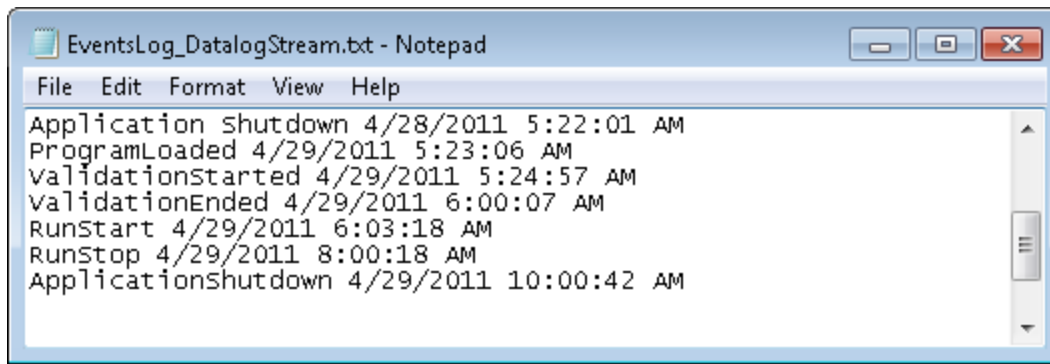
Using the Tester Utilization Data Consumer in IG-XL

Once the SysEventTracker and the EventFileNameContainer data DLLs have been built, they are ready for integration into an IG-XL test program. Data consumers need to be loaded and enabled for a particular data stream. In this case, add the SysEventTracker to the default data stream. You can do this in the OnProgramLoaded interpose function. Note that you must also explicitly ask the system to generate system events:

Function OnProgramLoaded()

```
On Error Resume Next
' Add a reference to the datacontainer
Application.ActiveWorkbook.VBProject.References.AddFromFile _
    "C:\Datalog\Custom\EventFileNameContainer.tbl"
On Error GoTo errHandler
' Turn on System Events
TheExec.Datalog.Events.EnableAll
' Turn on datalogging
TheExec.Datalog.Setup.DatalogSetup.DatalogOn = True
' Load and enable the SysEventTracker Data Consumer
Call TheExec.Datalog.Setup.DataStreams.Item.Outputs.Custom.Create( _
    "C:\Datalog\Custom\SysEventTracker.dll", _
    "TesterUtilization.DataConsumer.SysEventTracker")
Call TheExec.Datalog.Setup.DataStreams.Item.Outputs.Custom.Enable
TheExec.Datalog.ApplySetup
' Create an Event File Name Container and send it to the datalog stream
Dim container As Object
Set container = CreateObject( _
    "TesterUtilization.CustomerContainer.EventFileNameContainer")
container.EventsFileName = "EventsLog_DatalogStream.txt"
container.StreamID = "DatalogStream_customConsumer"
TheExec.Datalog.SendData container
Set container = Nothing
Exit Function
errHandler:
    HandleExecLError "OnProgramLoaded"
End Function
```

[Here is a sample output of the custom tester utilization datalog:](#)



```
File Edit Format View Help
Application Shutdown 4/28/2011 5:22:01 AM
ProgramLoaded 4/29/2011 5:23:06 AM
ValidationStarted 4/29/2011 5:24:57 AM
ValidationEnded 4/29/2011 6:00:07 AM
RunStart 4/29/2011 6:03:18 AM
RunStop 4/29/2011 8:00:18 AM
ApplicationsShutdown 4/29/2011 10:00:42 AM
```

Tester Utilization Datalog

The log indicates that the tester was idle for 24 hours during this two-day period. In addition, there two idle hours waiting for someone to pick up the tested devices.

<	>
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dISDKUsing.2.01

<

>

Using Datalog SDK

Using Datalog SDK

This section describes creation of data consumer and data container DLLs and their inclusion and usage in IG-XL.

The following topics are available:

- Prerequisites
- Setting Up a Custom DataConsumer
- DataConsumer Template Code
- Setting Up a DataContainer
- Custom DataContainer Code
- Using DataConsumer and DataContainer in IG-XL
- Debugging a Custom DataConsumer
- Debugging a DataContainer
- FAQs

< >		
	2015 Teradyne, Inc. All rights reserved.	

dISDKUsing.2.02

<	>
-----------------	-----------------

Using Datalog SDK : Prerequisites

Prerequisites

The following are prerequisites for working with custom data consumer and data container projects:

- | |
|---|
| <ul style="list-style-type: none">The currently supported version of Visual Studio must be installed. For the currently supported version, see the current release notes under the heading "System Requirements." |
| <ul style="list-style-type: none">Developers should have basic Visual C# programming skills to accelerate development of custom data consumers and containers. For information on C# programming concepts, consult the MSDN documentation at this site: |

[http://msdn.microsoft.com/en-us/library/67ef8sbd\(VS.80\).aspx](http://msdn.microsoft.com/en-us/library/67ef8sbd(VS.80).aspx)
Or go to the MSDN site ([msnd.microsoft.com](http://msdn.microsoft.com)) and search for "C# Programming Guide." Make sure the version is for the version of Visual Studio that you have installed.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dISDKUsing.2.03

<	>
-----------------	-----------------

Using Datalog SDK : Setting Up a Custom DataConsumer

Setting Up a Custom DataConsumer

This section describes creating and using a Custom DataConsumer with the following steps:

- | | |
|---|---------------------------------------|
| • | Install the DataConsumer Template |
| • | Create the DataConsumer Project |
| • | Set the DataConsumer Project Settings |

<	>
-----------------	-----------------

dISDKUsing.2.04

<	>
-----------------	-----------------

Using Datalog SDK : Setting Up a Custom DataConsumer : Install the DataConsumer Template

Install the DataConsumer Template
To install the DataConsumer template:

- | | |
|----|--|
| 1. | Go to the folder igxl-install-dir\src\Datalog\Templates. |
| 2. | Copy the file DataConsumerTemplate.zip to the folder My Documents\Visual Studio <version-number>\Templates\ProjectTemplates\Visual C#. |

[Copy the](#) .zip file itself. Do not unzip the file.

- | | |
|----|--|
| 3. | Go to Create the DataConsumer Project. |
|----|--|

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dISDKUsing.2.05

<

>

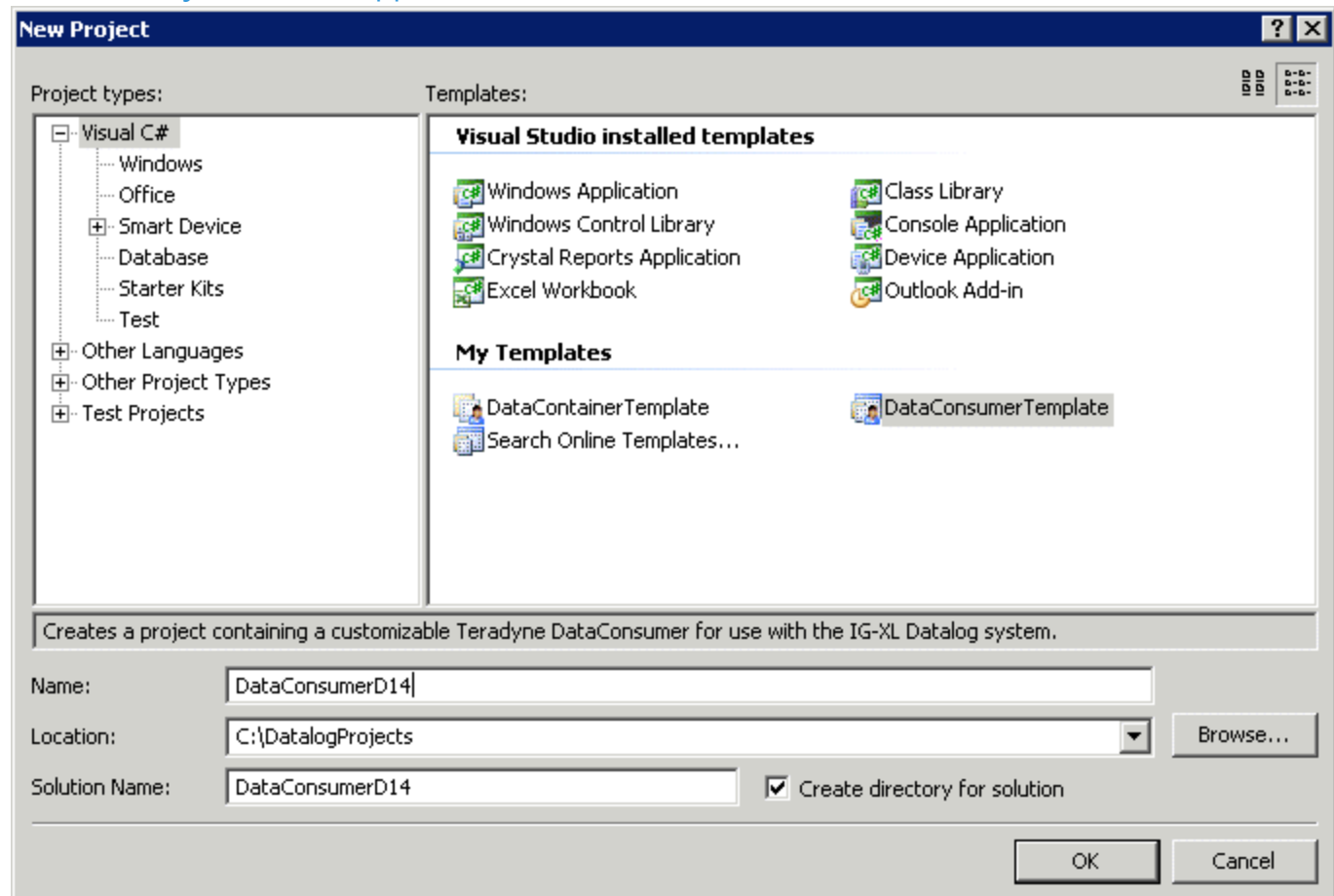
[Using Datalog SDK : Setting Up a Custom DataConsumer](#) : Create the DataConsumer Project

Create the DataConsumer Project

To create the DataConsumer project:

1. [Open the Visual Studio editor and select File>New>Project.](#)

The New Project window appears:



New Project Window

2. Fill out this window as follows:

- From the Project types panel on the left, select Visual C#.
- In the Templates panel on the right, under My Templates, select Data Consumer Template.
- In the boxes at the bottom of the window, enter a project name in the Name: text box. By default this is also used as the solution name.
- Click the Browse button next to the Location: text box and browse to a location.
- Make sure the box Create directory for solution is checked.
- Click OK.

The Visual Studio IDE now appears. The Solution Explorer panel on the left shows the files in the project.

3. Go to Set the DataConsumer Project Settings.

			
2015 Teradyne, Inc. All rights reserved.			

dISDKUsing.2.06



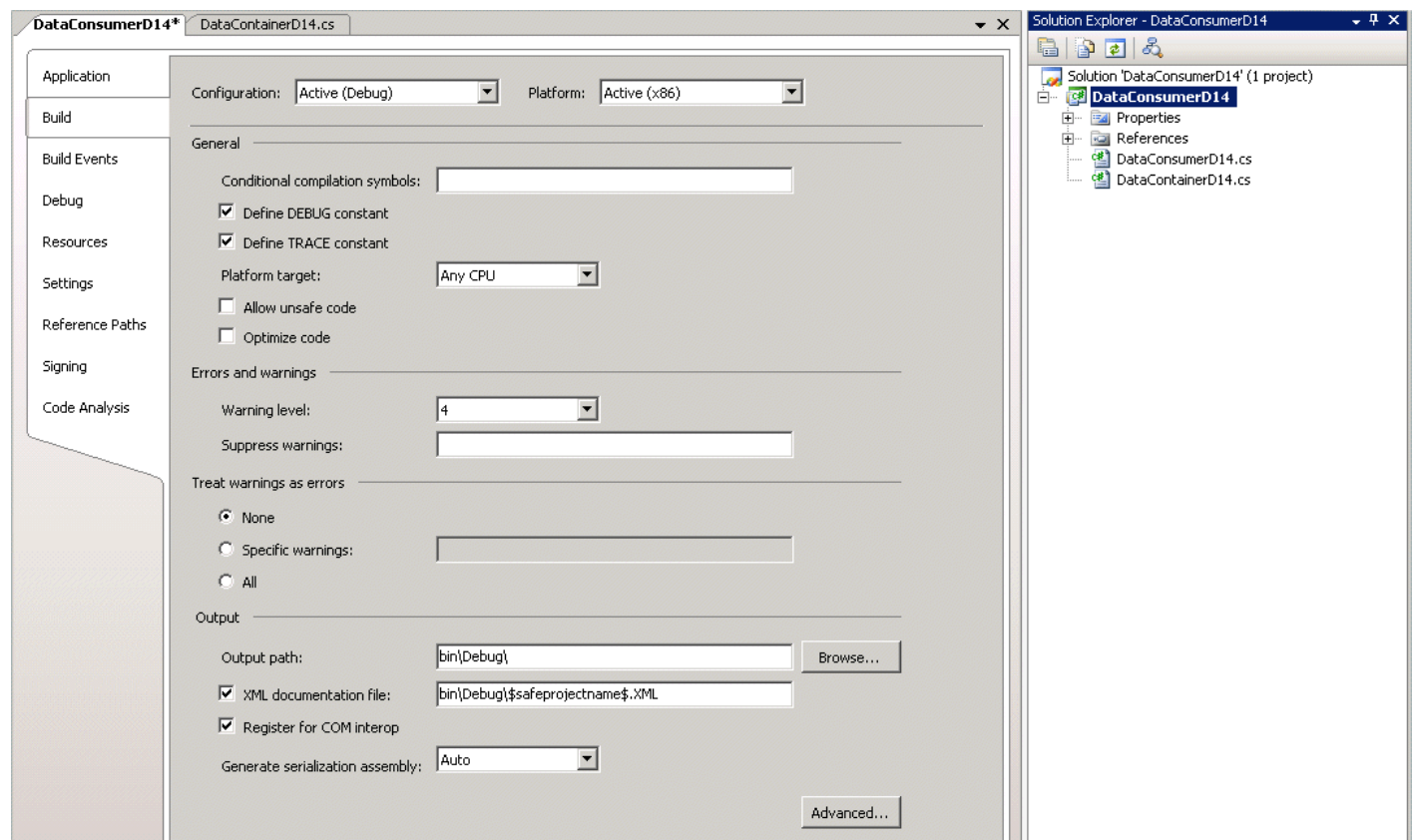
[Using Datalog SDK : Setting Up a Custom DataConsumer](#) : Set the DataConsumer Project Settings

Set the DataConsumer Project Settings

The DataConsumer project must have certain settings. Do the following:

1. [Right-click the project name in the Solution Explorer \(the node under the top-level Solution\)](#) and select Properties from the context menu.

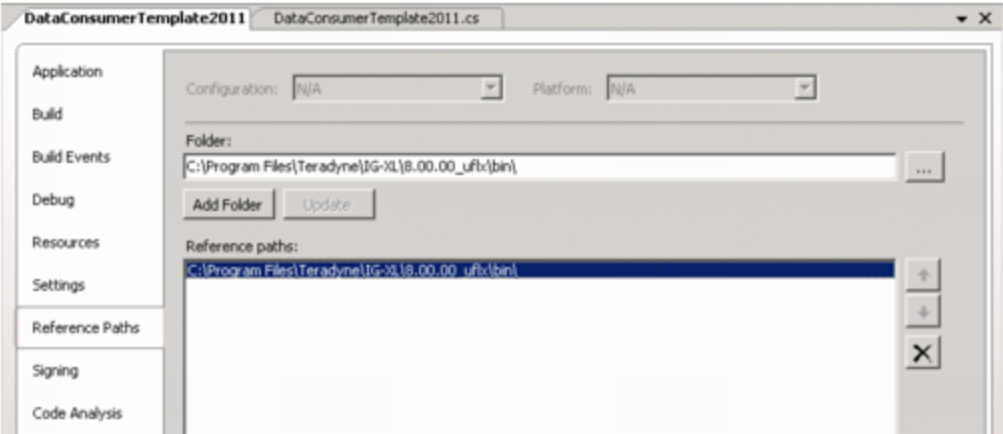
[Continuing the example from above, this is](#) DataConsumer. Properties appear as tabs along the left side of the IDE:



Solution Explorer Showing Properties Tabs

2. [Select the](#) Build tab. If you are creating the DataContainer class in the DataConsumer assembly, make sure that the box Register for COM Interop is checked. This is required to register the assembly for COM interop and to generate the type library.

3. [Select the](#) Reference Paths tab:



[Solution Explorer, Reference Paths Tab](#)

4. [Use the ellipsis button \(...\)](#) next to the Folder: text box to bring up the Windows browser.

5. [Navigate to the](#) <IG-XL Install>\bin folder.

6. [Click](#) Open. The Add Folder button is now enabled.

7. [Click](#) Add Folder to add the bin folder to the Reference paths: window.

This reference path provides the path to the folder from which the custom data consumer project can load the reference DLLs.

dISDKUsing.2.07

<	>
---	---

Using Datalog SDK : DataConsumer Template Code

DataConsumer Template Code

The DataConsumer.cs file provides the actual template for creating the custom consumer.
The steps in creating the consumer are:

- [Building the DataPacket Dictionary](#)
- [Handling DataPackets](#)
- [Exception Handling](#)
- [Final Cleanup and Processing](#)
- [Building the Custom DataConsumer Project](#)

You provide a packet handler for each packet to be handled. Your needs determine how data packets are handled. You handle only the packets that you care about, and you can ignore the rest. This flexibility is the primary advantage of custom consumers. For example, if you want summary details to be logged in the created custom consumer, you can handle only summary related packets. Similarly, if you need system events to be logged in the custom consumer, you can handle only system event related packets.
Information on data packets is available in IG-XL Help in the Teradyne.Igxl.Utilities.Datalog.DataPackets namespace:

UltraFLEX User Manuals - IG-XL V8.00.00_uflx

Hide Locate Back Forward Home Print Options

Contents Index Search Favorites

- Safety Guide and Operator Instructions
- IG-XL Programming Concepts
- UltraFLEX Instruments
- IG-XL Tools
- VBT Syntax Reference
- Logical MicroCode (LMC) Reference
- Pattern Language
- DSP and DSPWave Reference
- Concurrent Test
- Characterization
- Production Controls
- Handler/Prober Interface
- Calibration
- DIB Design Guidelines
- Hardware Specifications
- Datalog SDK
 - Using the Datalog SDK
 - Datalog SDK Reference
 - Teradyne.Igxl.Utilities.Datalog Namespace
 - Teradyne.Igxl.Utilities.Datalog.DataConsumers Namespace
 - Teradyne.Igxl.Utilities.Datalog.DataConsumers.Attributes Namespace
 - Teradyne.Igxl.Utilities.Datalog.DataConsumers.Setup Namespace
 - Teradyne.Igxl.Utilities.Datalog.DataConsumers.Summary Namespace
 - Teradyne.Igxl.Utilities.Datalog.DataPackets Namespace

Collapse All Code: All

Teradyne Datalog SDK

Teradyne.Igxl.Utilities.Datalog.DataPackets Namespace

[Send Feedback](#)

The DataPackets namespace provides the definitions for all DataPacket classes used by IG-XL. A DataPacket is literally a packet of information created by IG-XL and used to convey data to the DataConsumers for datalogging. All DataPacket inherit from the base DataPacket class.

DataPacket
Class

Properties

- HasStatistics
- Statistics

BeginProgramSection...
Class
↳ DataPacket

EndProgramSection...
Class
↳ DataPacket

IG-XL Help, Location of Data Packets Topic

Sample code to build a data packet dictionary and to handle data packets is provided in the [SampleDataConsumer](#) project. The sample project is located in the <IG-XL Install>\src\Datalog\Templates folder.

<	>
---	---

2015 Teradyne, Inc. All rights reserved.		
--	--	--

dISDKUsing.2.08

<	>
-----------------	-----------------

Using Datalog SDK : [DataConsumer Template Code](#) : Building the DataPacket Dictionary

Building the DataPacket Dictionary

The following sample code shows how to build the data packet dictionary:

```
protected override void BuildDataPacketDictionary(ISearchCollection<Type,
    DataConsumer.ProcessPacketHandler> outDictionary)
{
    //Process System Command Packet
    outDictionary.Add(typeof(CustomEventPacket), HandleCustomEventPacket);
    //Process Header Packet
    outDictionary.Add(typeof(OpenOutputsPacket), HandleOpenoutputPacket);
}
```

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dISDKUsing.2.09

<	>
-----------------	-----------------

Using Datalog SDK : DataConsumer Template Code : Handling DataPackets

Handling DataPackets

This section and the following ones describe packets sent from the datalogging subsystem to the custom data consumers. You can query for an incoming data packet and access its properties and methods. See the DataPacket library documentation for more information. The path to the library is shown under DataConsumer Template Code.

Datalog supports the following types of data packets:

- | | |
|---|---------------------------|
| • | Custom Event Packet |
| • | System Event Packets |
| • | Synchronous Data Packets |
| • | Asynchronous Data Packets |

<	>
-----------------	-----------------

dlSDKUsing.2.10



Using Datalog SDK : [DataConsumer Template Code](#) : Custom Event Packet

Custom Event Packet

This is sample code for handling [CustomEventPacket](#):

```
protected virtual void HandleCustomEventPacket(DataPacket inPacket)
{
    private StreamWriter eventLogFile;
    if (eventLogFile == null)
    {
        eventLogFile = new StreamWriter(eventLogFile, true);
        eventLogFile.AutoFlush = true;
    }
    try
    {
        CustomEventPacket syspacket = (CustomEventPacket)inPacket;
        if (syspacket != null)
        {
            if (eventLogFile != null)
            {
                eventLogFile.WriteLine("Custom Event Major Id:" + syspacket.MajorId);
                eventLogFile.WriteLine("Custom Event Minor Id:" + syspacket.MinorId);
                eventLogFile.WriteLine("Custom Event Data:" + syspacket.Data);
                eventLogFile.WriteLine();
            }
        }
    }
}
```

```
catch (Exception exp
{
    throw new CustomConsumerException("SampleDataConsumer:CustomEvent Packet error"+
exp.ToString());
}
}
```

dISDKUsing.2.11

<

>

Using Datalog SDK : DataConsumer Template Code : System Event Packets

System Event Packets

System events are classified into the following:

- BaseSystemEvents Packets
- ExtendedSystemEvents Packets

BaseSystemEvents Packets

This is sample code for handling a BaseSystemEvents packet:

```
protected virtual void HandleSystemEventPacket(DataPacket inPacket)
{
    private StreamWriter eventsFile;
    if (eventsFile == null)
    {
        eventsFile = new StreamWriter(eventsFile, true);
        eventsFile.AutoFlush = true;
    }
    try
    {
        SystemEventPacket syspacket = (SystemEventPacket)inPacket;
        if (syspacket != null)
        {
            if (eventsFile != null)
            {
                switch (syspacket.EventId)
```

```
{  
case SystemEventType.ProgramLoaded:  
    eventsFile.WriteLine("SystemEventType.ProgramLoaded");  
    break;  
case SystemEventType.ValidationStart:  
    eventsFile.WriteLine("SystemEventType.ValidationStart");  
    break;  
case SystemEventType.RunStart:  
    eventsFile.WriteLine("SystemEventType.RunStart");  
    break;  
case SystemEventType.RunStop:  
    eventsFile.WriteLine("SystemEventType.RunStop");  
    break;  
case SystemEventType.StartWafer:  
    eventsFile.WriteLine("SystemEventType.StartWafer");  
    break;  
case SystemEventType.EndWafer:  
    eventsFile.WriteLine("SystemEventType.EndWafer");  
    break;  
case SystemEventType.TDRCalibrationStart:  
    eventsFile.WriteLine("SystemEventType.TDRCalibrationStart");  
    break;  
case SystemEventType.TDRCalibrationEnd:  
    eventsFile.WriteLine("SystemEventType.TDRCalibrationEnd");  
    break;  
case SystemEventType.EndLot:  
    eventsFile.WriteLine("SystemEventType.EndLot");  
    break;  
case SystemEventType.ApplicationShutdown:  
    eventsFile.WriteLine("SystemEventType.ApplicationShutdown");  
    break;  
case SystemEventType.CalibrationStart:  
    eventsFile.WriteLine("SystemEventType.CalibrationStart");  
    break;  
}  
eventsFile.WriteLine();  
}
```

```
}  
catch (Exception exp)  
{  
    throw new CustomConsumerException ("SampleDataConsumer:System Packet handling error "  
+exp.ToString());  
}  
}
```

ExtendedSystemEvents Packets

Extended system event packets include:

- ValidationEndEventPacket

- CalibrationDoneEventPacket

- CalibrationPowerupResetEventPacket

- ChangeModeEventPacke

- TDRCalibrationDoneEventPacket

This is sample code for handling a ValidationEndEventPacket:

```
protected virtual void HandleValidationEndPacket(DataPacket inPacket)  
{  
    private StreamWriter eventsFile;  
    if (eventsFile == null)  
    {  
        eventsFile = new StreamWriter(eventsFile, true);  
        eventsFile.AutoFlush = true;  
    }  
    try  
    {  
        ValidationEndEventPacket syspacket = (ValidationEndEventPacket)inPacket;  
        if (syspacket != null)  
        {  
            if (eventsFile != null)  
            {  
                switch (syspacket.Passed)
```

```
{
case true:
eventsFile.WriteLine("Validation Passed");
break;
case false:
eventsFile.WriteLine("Validation Failed");
break;
}
eventsFile.WriteLine("Event Time Stamp " + new DateTime(syspacket.TimeStamp).ToString());
}
}
}
catch (Exception exp)
{
throw new CustomConsumerException ("SampleDataConsumer:Validation End Event Packet
handling error " + exp.ToString());}
```


dISDKUsing.2.12



Using Datalog SDK : DataConsumer Template Code : Synchronous Data Packets

Synchronous Data Packets

Validation Data and Apply Setup Data are synchronous data packets. You can access these data packets when the following asynchronous data packets are received:

- Validation data -> SystemCommandPacket with System command as Validation
- ApplySetup data -> SystemCommandPacket with System command as ApplySetup

```
protected virtual void HandleSystemCommandPacket(DataPacket inPacket)
{
    try
    {
        SystemCommandPacket sysCmdPacket = (SystemCommandPacket)inPacket;
        if (sysCmdPacket != null)
        {
            switch (sysCmdPacket.CommandId)
            {
                case SystemCommandType.ApplySetup:
                    ProcessSetupData();
                    break;
                case SystemCommandType.Validation:
                    ProcessValidationData();
                    break;
            }
        }
    }
}
```

```
}  
catch (Exception exp)  
{  
    throw new CustomConsumerException ("SampleDataConsumer:System Command  
Packet                      handling error " + exp.ToString());  
}  
}  
void ProcessValidationData()  
{  
    ValidationData validationObject = base.ValidationData;  
    if (validationObject!= null)  
    {  
        //Any data from the validation object can be obtained and processed as required.  
        unsigned int startTime = validationObject.StartTime;  
        string testInstance = validationObject.testInstance;  
        string channelMap = validationObject.channelMap;  
        int siteCount = validationObject.siteCount;  
    }  
}  
void ProcessSetupData()  
{  
    StreamSetup streamSetup = base.SetupData;  
    If (streamSetup != null)  
    {  
        //Any data from the setup object can be obtained and processed as required.  
        SharedSetup sharedSetup = streamSetup.sharedSetup;  
        ParametricSetups parametricSetup = streamSetup.parametricSetup;  
        FunctionalSetups functionalSetup = streamSetup.functionalSetup;  
        STDFOutputSetup stdfSetup = streamSetup. stdfSetup;  
    }  
}
```



dlSDKUsing.2.13



Using Datalog SDK : DataConsumer Template Code : Asynchronous Data Packets

Asynchronous Data Packets

Asynchronous data packets include all remaining data packets, that is, those not listed under Synchronous Data Packets.

This is sample code for handling one of the asynchronous data packets,

FunctionalTestShortResultPacket:

```
protected virtual void HandleFunctionalTestShortResultPacket(DataPacket inPacket)
{
    private StreamWriter eventLogFile;
    if (customFile == null)
    {
        customFile = new StreamWriter(customFile, true);
        customFile.AutoFlush = true;
    }
    try
    {
        FunctionalTestShortResultPacket funcTest = (FunctionalTestShortResultPacket)inPacket;
        if (funcTest != null)
        {
            if (customFile != null)
            {
                if (CurTestType != CurrentTestType.FunctionalShort)
                {
                    customFile.WriteLine();
                    customFile.Write("Number    ");
                    customFile.Write("Site    ");
                }
            }
        }
    }
}
```

```
        customFile.Write("Test Name      ");
        customFile.Write("Pattern      ");
        customFile.Write("1st Failed Cycle ");
        customFile.Write("Total Failed Cycle ");
        customFile.WriteLine();
        customFile = CurrentTestType.FunctionalShort;
    }
    foreach (int site in funcTest.SiteNumbers)
    {
        customFile.Write("0" + "      ");
        customFile.Write(site.ToString() + "      ");
        customFile.Write(m_CurTestName.TestName + "      ");
        customFile.Write(funcTest.PatternName + "      ");
        customFile.Write("N/A" + "      ");
        customFile.Write("N/A" + "      ");
        customFile.WriteLine();
    }
}
}
}
}
catch (Exception exp)
{
    throw new CustomConsumerException ("SampleDataConsumer:FunctionalTestShortResult Packet
handling error " + exp.ToString());
}
}
```

dISDKUsing.2.14

<	>
----------------------	----------------------

Using Datalog SDK : DataConsumer Template Code : Exception Handling

Exception Handling

Code written inside the `ErrorAlarmPacket` and `DatalogCommentPacket` handlers must be encapsulated in a try/catch block. Allowing unhandled exceptions to occur within these handlers may lead to an infinite message loop between the Excel and Datalog Server processes. Specifically, this possibility exists for any `DataPacket` handler written for any `DataPacket` inheriting from and including the `DatalogCommentPacket`. Currently, this applies to only two packet types. The process is as follows:

- An unhandled exception from any handler method of a customer data consumer returns an error to the Datalog Service.
- In response to this error, the Datalog Service generates an `ErrorAlarmPacket` that is sent to all consumers.
- If an unhandled exception occurs in the `ErrorAlarmPacket` handler of a customer data consumer, that error generates another message back to the Datalog Service, resulting in another `ErrorAlarmPacket`.

This cause-and-effect loop results in an infinite loop until the `ErrorAlarmPacket` handler no longer produces an exception. Preventing an unhandled exception from leaving the `ErrorAlarmPacket` or `DatalogCommentPacket` handlers prevents this infinite loop from occurring.

<	>
----------------------	----------------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dISDKUsing.2.15

<	>
---	---

Using Datalog SDK : DataConsumer Template Code : Final Cleanup and Processing

Final Cleanup and Processing

Performing final processing and releasing all resources used is done in `FinalizeCleanupAndProcessing`. To do this, you must override the method `FinalizeCleanupAndProcessing()` of the `DataConsumer` class.

Here is sample code for the method:

```
protected override void FinalizeCleanupAndProcessing()
{
    if (eventLogFile != null)
    {
        eventLogFile.Close();
    }
    base.FinalizeCleanupAndProcessing();
}
```

After coding, build the project and generate the custom consumer DLL. See [Building the Custom DataConsumer Project](#). The custom consumer DLL and classes are now ready to be loaded by IG-XL.

<	>
---	---

dISDKUsing.2.16

<	>
---	---

Using Datalog SDK : [DataConsumer Template Code](#) : Building the Custom DataConsumer Project

Building the Custom DataConsumer Project

To build the custom DataConsumer project, select the [Build](#) menu in the Visual Studio IDE and then select the [Build Solution](#) menu item. Once the project is built successfully, the custom DataConsumer DLL is generated in the folder <Project Path>\Debug or <Project Path>\Release, based on the Configuration selected. If any errors occur, they are listed in the Output Window and the Error List Window.

Additional information on building C# projects is available at the following MSDN topic:
[http://msdn.microsoft.com/en-us/library/5tdasz7h\(v=VS.80\).aspx](http://msdn.microsoft.com/en-us/library/5tdasz7h(v=VS.80).aspx)

Or log onto msdn.microsoft.com and search for "Prepare and Manage Builds." Make sure that you are looking at the Visual Studio 2005.

<	>
---	---

2015 Teradyne, Inc. All rights reserved.		
--	--	--

dISDKUsing.2.17

<

>

Using Datalog SDK : Setting Up a DataContainer

Setting Up a DataContainer

This section describes how to create and use a DataContainer. You can create a DataContainer either as a stand-alone project or as part of an existing DataConsumer assembly. Both methods are described in the following sections.

The following steps are described:

- [Install the DataContainer Template](#)
- [Create a Stand-Alone DataContainer](#)
- [Add the DataContainer as a New Item](#)
- [Set the DataContainer Project Settings](#)

<

>

dISDKUsing.2.18

<

>

Using Datalog SDK : Setting Up a DataContainer : Install the DataContainer Template

Install the DataContainer Template
To install the DataContainer template:

1. [Navigate to](#) igxl-install-dir\src\Datalog\Templates.
2. [Copy the file](#) DataContainerTemplate.zip to the folder My Documents\Visual Studio 2005\Templates\ItemTemplates\Visual C#.

[Copy the](#) .zip file itself. Do no unzip the file.

3. [For the next step, go to one of the following:](#)
- [Create a Stand-Alone DataContainer](#)

– [Add the DataContainer as a New Item](#) (if you are adding the data container to an existing DataConsumer project)

<

>

dlSDKUsing.2.19



[Using Datalog SDK : Setting Up a DataContainer](#) : Create a Stand-Alone DataContainer

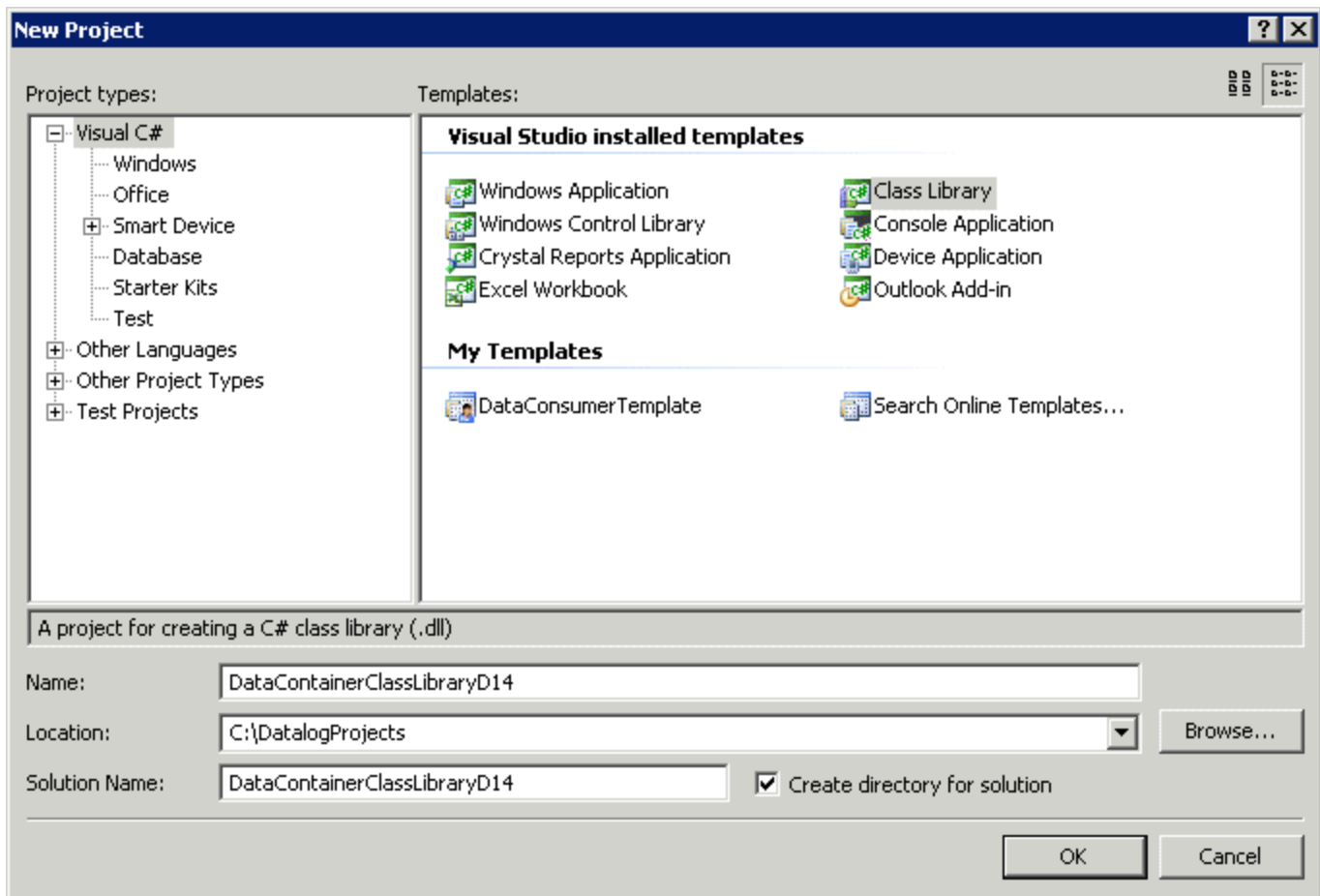
Create a Stand-Alone DataContainer

This section describes how to create a stand-alone data container. You can also create a data container as part of an existing DataConsumer assembly. To create the data container in an existing DataConsumer project, go to [Add the DataContainer as a New Item](#).

To create a stand-alone data container, follow these steps.

1. [Open the Visual Studio 2005 editor and select File>New>Project](#).

The New Project window appears:



New Project Window

2. Fill out this window as follows:

- From the Project types panel on the left, select Visual C#.
- In the Templates panel on the right, under Visual Studio installed templates, select Class Library.
- In the boxes at the bottom of the window, enter a class library name.
- Use the Browse button to browse to a location.
- Keep the box Create directory for solution checked.
- Click OK.

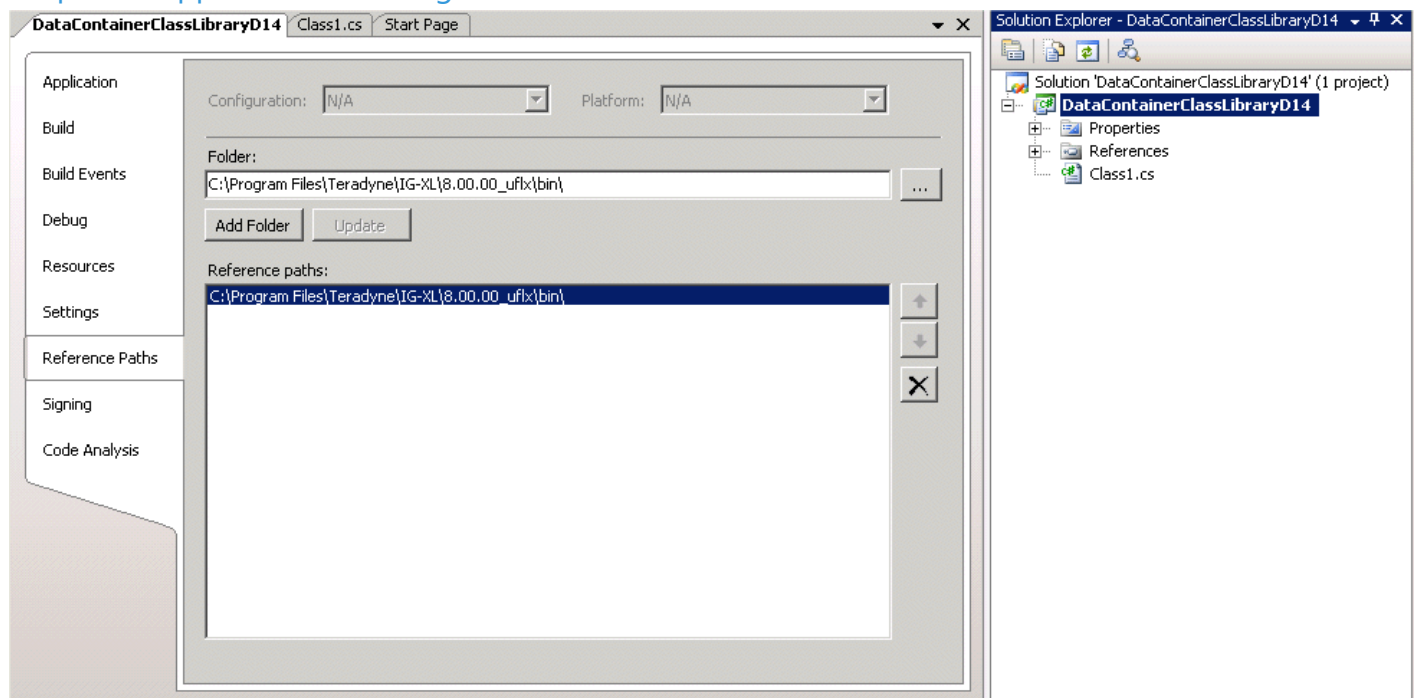
The Visual Studio IDE now appears. The Solution Explorer panel on the right shows the files in the data container.

The data container must have certain settings. To display the project properties, select the project name in the Solution Explorer. This is the node under the top-level Solution. In the example above, this is DataConsumer. Select this node, right-click, and select Properties from the context menu.

Do the following:

1. The DataConsumer project must have certain settings. Right-click the project name in the Solution Explorer (the node under the top-level Solution) and select Properties from the context menu.

Properties appear as tabs along the left side of the IDE:



Visual Studio IDE Showing Properties Tabs

2. Use the ellipsis button (...) next to the Folder: text box to bring up the Windows browser.
3. Navigate to the <IG-XL Install>\bin folder.
4. Click Open. The Add Folder button is now enabled.
5. Click Add Folder to add the bin folder to the Reference paths: window.

This reference path provides the path to the folder from which the data container project can load the reference DLLs.

6.

Go to Add the DataContainer as a New Item.

<

>

2015 Teradyne, Inc. All rights reserved.

dISDKUsing.2.20



[Using Datalog SDK : Setting Up a DataContainer](#) : Add the DataContainer as a New Item

Add the DataContainer as a New Item

At this point, you call up the Add New Item window to add the data container. To do this:

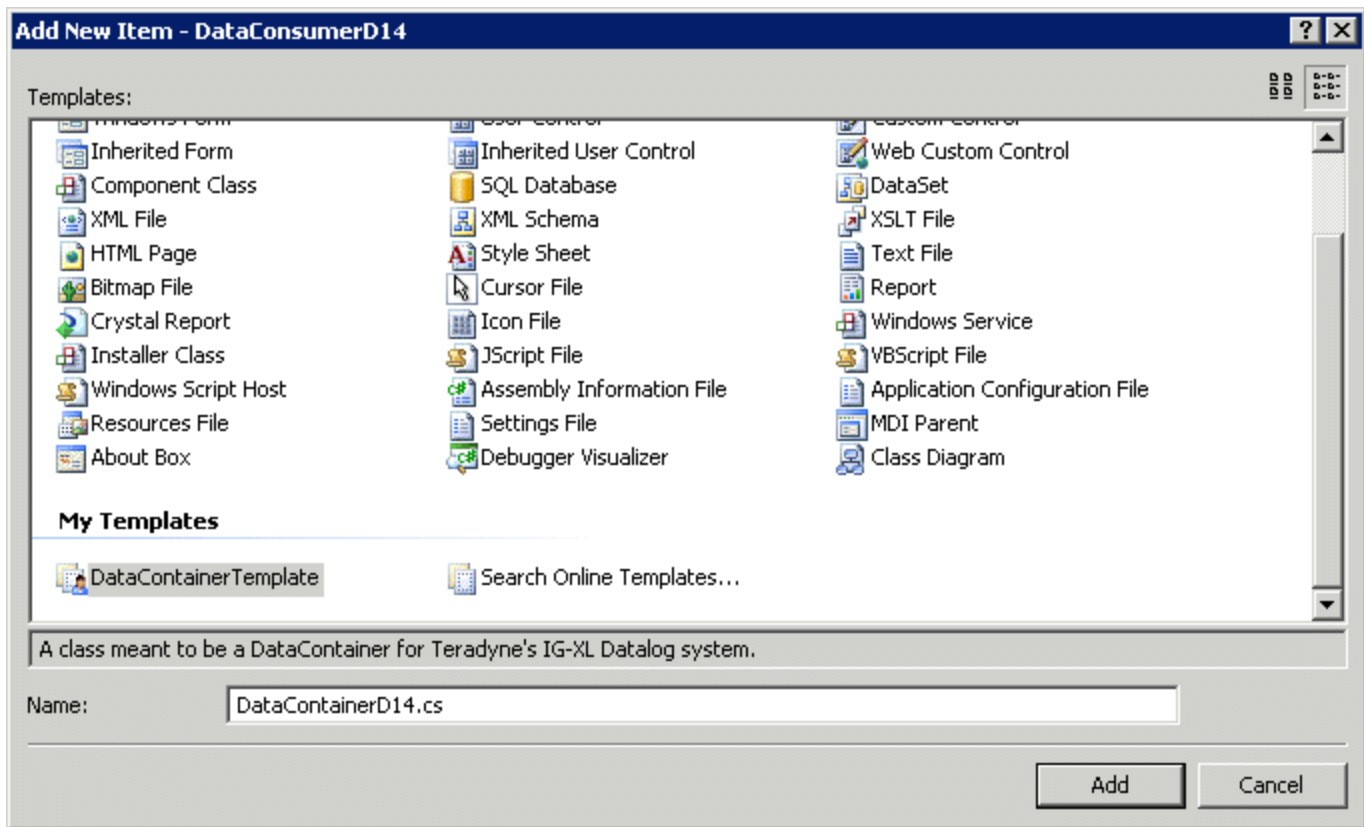
1. [Select the project in the Solution Explorer. The node to select is directly under the top-level Solution node:](#)

- [If you are adding the data container to an existing dataconsumer project, this is the data consumer project node.](#)

- [If you have created a stand-alone data container, this is the class library project node.](#)

2. [When you have selected the project, right-click and select Add>New Item.](#)

The Add New Item window appears:



Add New Item Window

3. Under My Templates at the bottom of the window, select DataContainerTemplate.
4. Name the item and click the Add button.

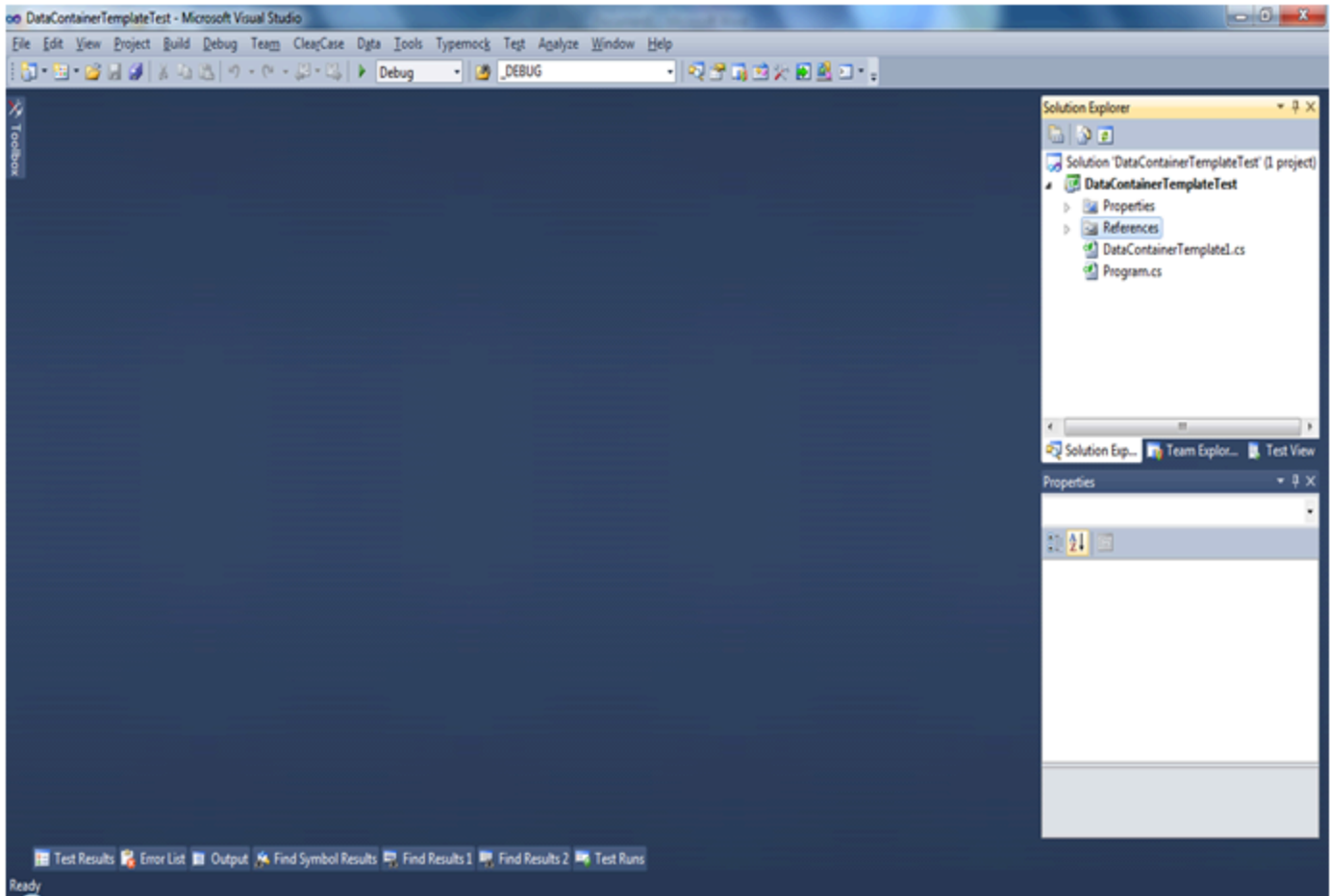
This displays the DataContainer.cs file. This template also provides references to required DLLs to create the custom container.

Additional Steps for Visual 2010 Only

If you are using Visual Studio 2010, you must perform the following additional steps when opening the template. This is because a property was introduced in Visual Studio 2010, and you must set the property to the correct value.

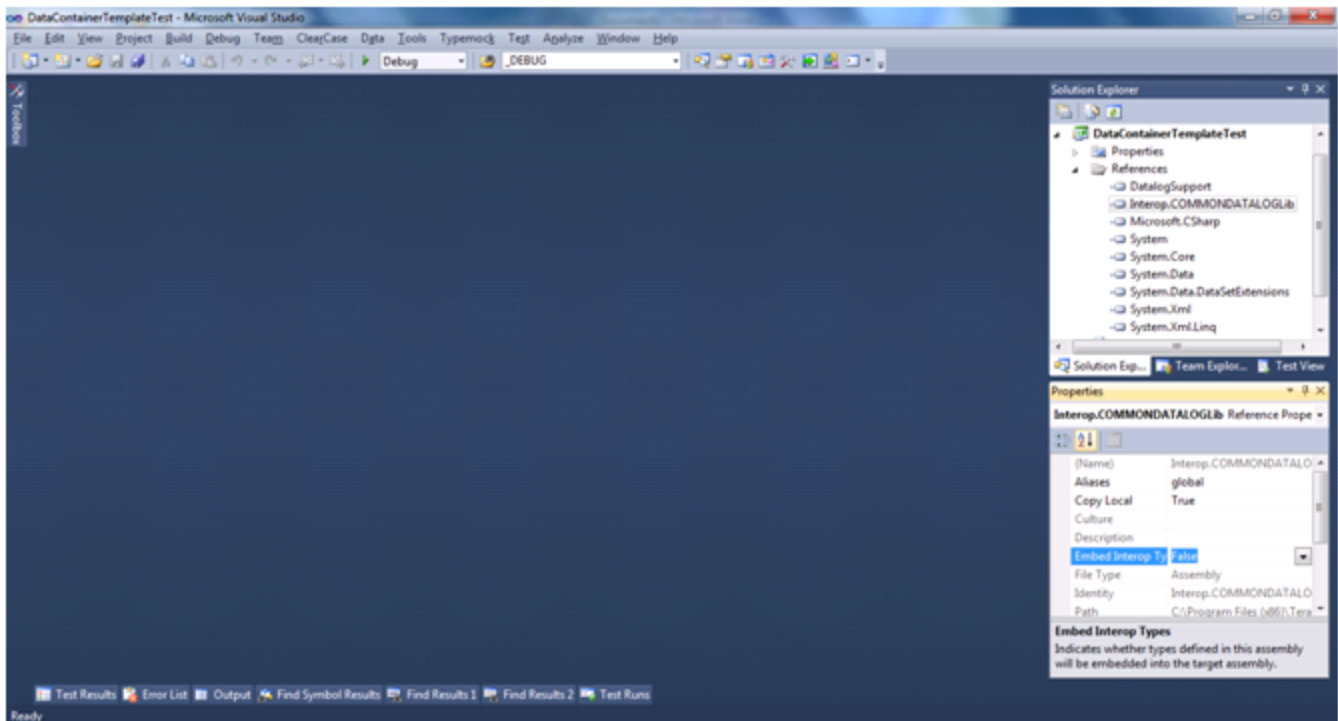
If you are using Visual Studio 2005, do not perform the following steps.

5. When you click the Add button, you get the following screen:



After Add, in Visual Studio 2010

6. In the Solution Explorer, expand the References item and select the reference Interop.COMMONDATALOG.lib:



References, in Visual Studio 2010

7. [In the Property window for Interop.COMMONDATALOG.lib](#), select the property Embed Interop Type and set its value to False. By default, the property is set to True, which would cause the compilation to fail. When you set it to False, the compilation will pass.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dlsdkUsing.2.21



Using Datalog SDK : Setting Up a DataContainer : Set the DataContainer Project Settings

Set the DataContainer Project Settings

The library must have certain settings. To do this, you set project properties:

1. Right-click the project name in the Solution Explorer and select Properties from the context menu. The node to select is directly under the top-level Solution node:

- If you are adding the data container to an existing dataconsumer project, this is the data consumer project node.

- If you have created a stand-alone data container, this is the class library project node.

Properties appear as tabs along the left side of the IDE.

2. Select the Application tab.

3. On the tab, click the Assembly Information... button.

The Assembly Information window appears:

Assembly Information

Title: DataContainerClassLibraryD14

Description:

Company: Teradyne, Inc

Product: DataContainerClassLibraryD14

Copyright: Copyright © Teradyne, Inc 2011

Trademark:

Assembly Version: 1 0 0 0

File Version: 1 0 0 0

GUID: 5b9f57ae-6db9-4d22-adb6-b8953c0acd9e

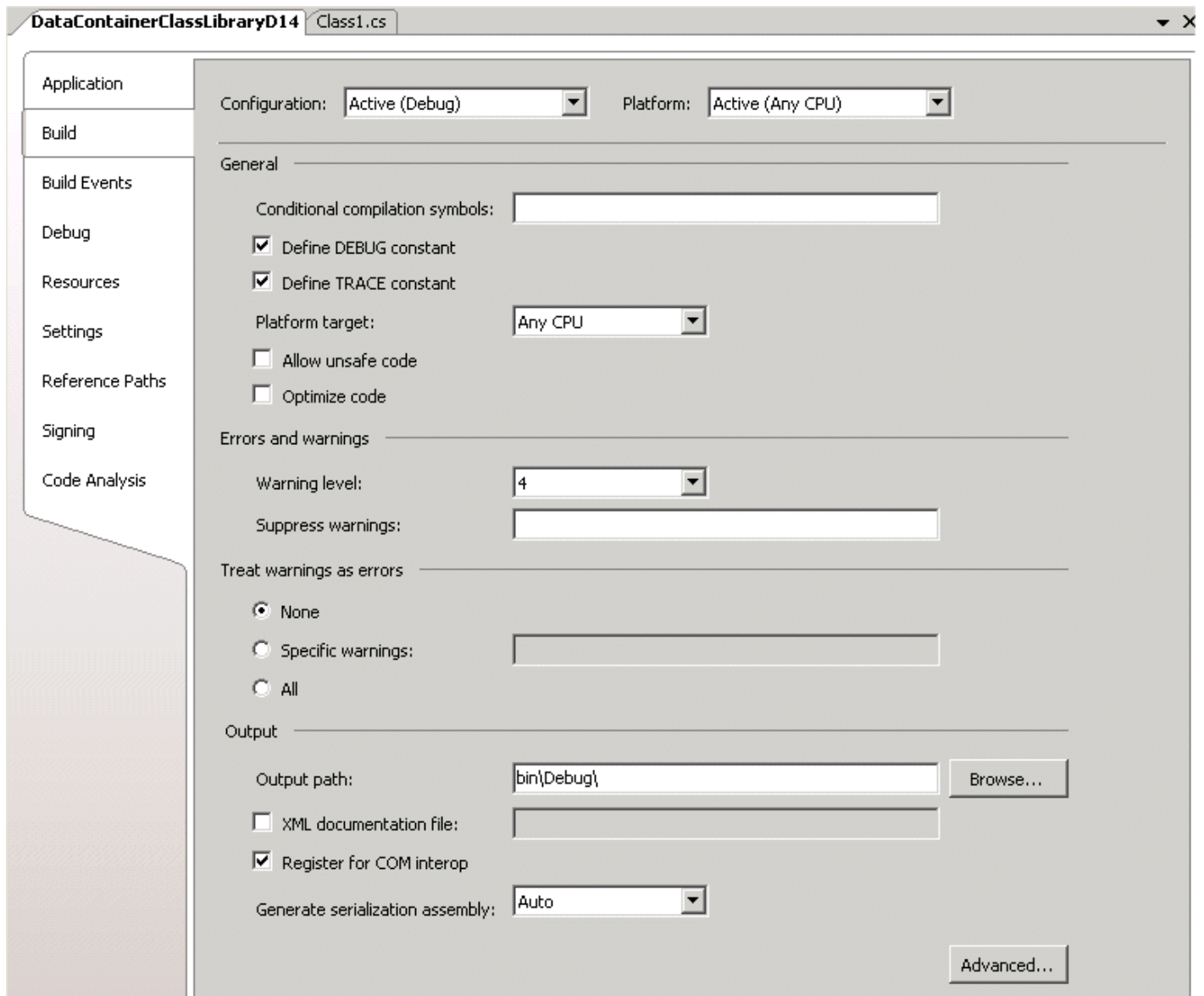
Neutral Language: (None)

☒ Make assembly COM-Visible

OK Cancel

Assembly Information Window

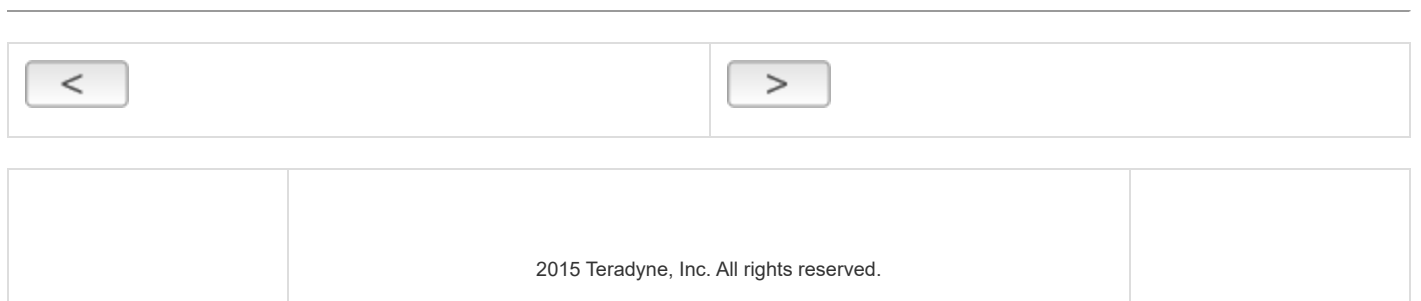
4. [Check the box](#) Make assembly COM-Visible. If the box is not checked, you cannot load or set the data container object until the DLL is registered using the command line `ateregasm /tlb <filename.dll>`. See [Registering and Unregistering the DataContainer DLL](#).
5. [On the IDE, select the tab for the](#) Build property:



Visual Studio IDE, Build Property Tab

6. [Check the box](#) Register for COM Interop.

This registers the assembly for COM interop and generates the type library (*.tlb file).



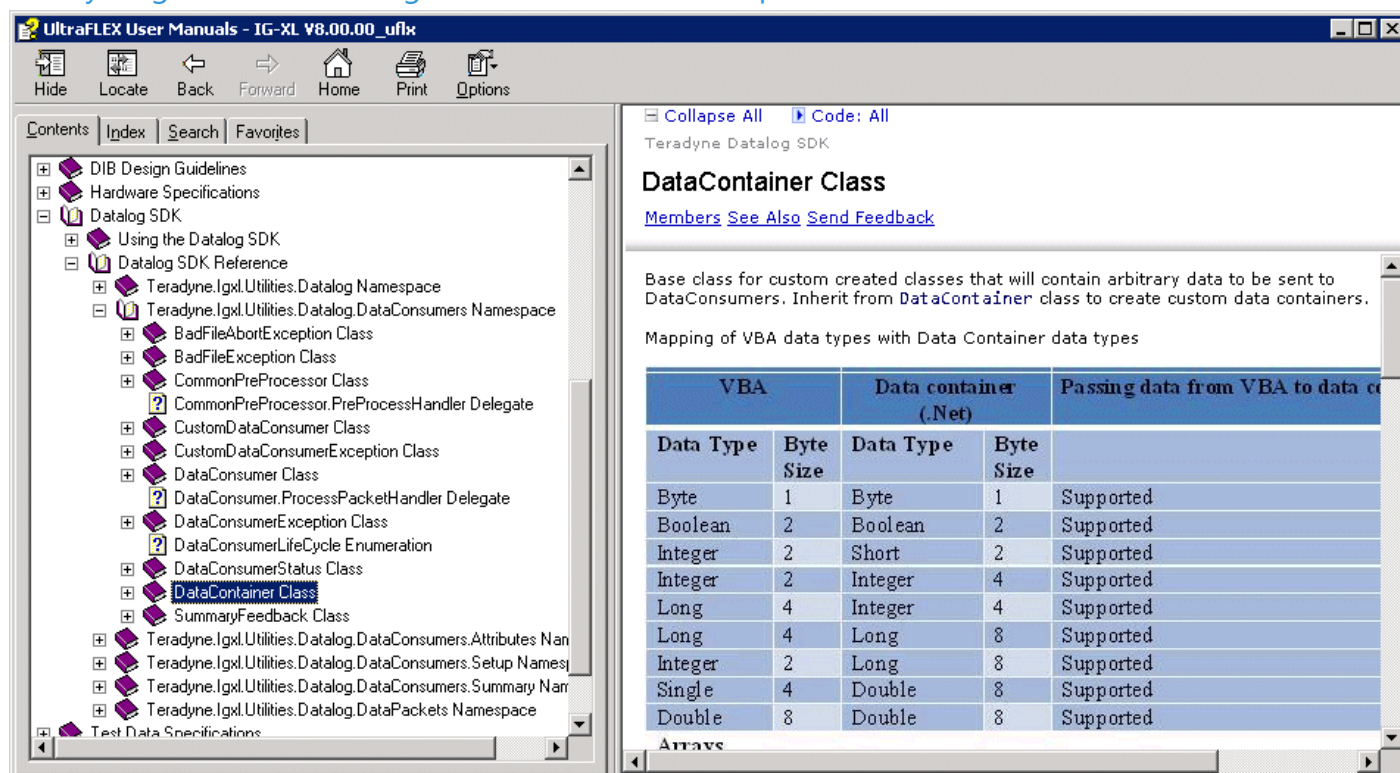
dlSDKUsing.2.22



Using Datalog SDK : Custom DataContainer Code

Custom DataContainer Code

You can create a custom data container to send data to custom consumers. The supported data types are described here. The reference documentation for the DataContainer class has a list of data types supported by Data Container. See this topic in the online IG-XL Help: [Teradyne.Iqxl.Utilities.Datalog.DataConsumers Namespace.DataContainer Class](#)



[DataContainer Class Topic in IG-XL Help](#)

The following topics are available:

- Setting Up the DataContainer Interface and Class

- [Handling Custom DataContainer Data](#)
- [Building the DataContainer Project](#)

<

>

2015 Teradyne, Inc. All rights reserved.

dlsdkUsing.2.23



Using Datalog SDK : Custom DataContainer Code : Setting Up the DataContainer Interface and Class

Setting Up the DataContainer Interface and Class

The primary concern when developing a project is the visibility of the container class members to the VBA. The DataContainerTemplate does not provide IntelliSense to access its methods and members from VBA. To make them visible to VBA, you can use the "AutoDual" attribute of a class. However, using this attribute also exposes all managed members of the class.

A better option is to create an interface that is COM-visible with all members and implement this interface in the class. While creating the interface, make sure that the interface is public and that the name of the interface is the same as that of the class name but preceded by "_". For example, if the class name is "CustomDataContainer" the interface name should be "_ CustomDataContainer".

See the SampleDataContainer project located in this folder:

<IG-XL Install>\src\Datalog\Templates

Here is sample code for the data container interface and data container class:

```
public interface _SampleContainer
{
    int IntegerValue { get; set; }
    int[] IntegerArray { get; set; }
}

public class SampleContainer : DataContainer, _SampleContainer
{
    protected int _intValue = 0;
    protected int[] _intArray = null;
    public int IntegerValue
    {
        set
```



```

    {
        _intValue = value;
    }
    get
    {
        return _intValue;
    }
}
public int[] IntegerArray
{
    set
    {
        _intArray = value;
    }
    get
    {
        return _intArray;
    }
}
}

```

Note that you can only use the above method to set an array if the .Net assembly is linked dynamically to VB (Set object = CreateObject ("SampleDataContainer.SampleContainer"). If the assembly is statically linked to VB (by adding a reference to the SampleDataContainer.tlb file using VBA Tools>References), the following compiler error is returned while trying to access the array: Function or interface marked as restricted, or the function uses an Automation type not supported in Visual Basic

This is because VB version 6.0 cannot use the .NET method with the ParamArray parameter in Microsoft Visual Studio .NET. You can work around the problem by adding a separate method using a ref parameter that assigns the property to the private member. See the following link: <http://support.microsoft.com/kb/327084>

Here is sample code for the method using the ref parameter.

```

public void SetIntegerArray(ref int[] intArray)
{
    _intArray = intArray;
}

```

If the data container is created as a separate project, after coding, build the project and generate the data container DLL.

To use the data container DLL that is created, you must add the DLL as a reference to the custom data consumer project. If the data container is created as a part of the custom data consumer project, you are not required to add a reference to the data container DLL.

< >	
	2015 Teradyne, Inc. All rights reserved.

dISDKUsing.2.24



Using Datalog SDK : Custom DataContainer Code : Handling Custom DataContainer Data

Handling Custom DataContainer Data

Handle the DataContainerPacket in the custom data consumer project so that data sent in the container is logged by the custom consumer. Data sent in the container is received as a packet in the custom consumer, and this packet is handled by the DataContainerPacket handler in the custom consumer project and can be processed further.

The following sample code handles a DataContainerPacket in a custom consumer:

```
protected virtual void HandleDataContainerPacket(DataPacket inPacket)
{
    try
    {
        DataContainerPacket container = (DataContainerPacket)inPacket;
        if (container != null)
        {
            int intValue = container.IntegerValue;
            int[] intArray = container.IntegerArray;
            private StreamWriter dataContainerFile;
            if (dataContainerFile == null)
            {
                dataContainerFile = new StreamWriter(m_eventsFileName, true);
                dataContainerFile.AutoFlush = true;
            }
            dataContainerFile.WriteLine("Integer Value : " + intValue.ToString());
            if (intArray != null)
            {
                dataContainerFile.Write("Integer Arrays : ");
            }
        }
    }
}
```

```
        foreach (int value in intArray)
        {
            dataContainerFile.Write(value.ToString() + ",");
        }
        dataContainerFile.WriteLine();
    }
}
}
}
catch (Exception exp)
{
    throw new CustomConsumerException ("SampleDataConsumer:DataContainer Packet   handling
error " + exp.ToString());
}
}
```

Sample code to handle the `DataContainerPacket` is provided in the `SampleDataConsumer` project located in the folder `<IG-XL Install>\src\Datalog\Templates`.

		
	2015 Teradyne, Inc. All rights reserved.	

dISDKUsing.2.25

<	>
-----------------	-----------------

Using Datalog SDK : Custom DataContainer Code : Building the DataContainer Project

Building the DataContainer Project

To build the DataContainer project, select the Build menu in the Visual Studio IDE and then select the Build Solution menu item. Once the project is built successfully, the custom DataContainer DLL and the .tlb file are generated in the folder <Project Path>\Debug or <Project Path>\Release, based on the Configuration selected. If any errors occur, they are listed in the Output Window and the Error List Window.

Additional information on building C# projects is available at the following MSDN topic:
[http://msdn.microsoft.com/en-us/library/5tdasz7h\(v=VS.80\).aspx](http://msdn.microsoft.com/en-us/library/5tdasz7h(v=VS.80).aspx)

Or log onto msdn.microsoft.com and search for "Prepare and Manage Builds." Make sure that you are looking at the Visual Studio 2005.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dISDKUsing.2.26

<

>

Using Datalog SDK : Using DataConsumer and DataContainer in IG-XL

Using DataConsumer and DataContainer in IG-XL

After you create and build the custom dataconsumer and the datacontainer, the .Net assemblies can be integrated with IG-XL. This section describes the integration of DLLs in IG-XL. This section assumes that you have created an IG-XL test program with at least one test. The following topics are available:

- Registering and Unregistering the DataContainer DLL
- Loading a Custom DataConsumer
- Sending Custom Data in DataContainer
- Receiving Data in a DataContainer

<

>

dISDKUsing.2.27

<	>
-----------------	-----------------

Using Datalog SDK : Using DataConsumer and DataContainer in IG-XL : Registering and Unregistering the DataContainer DLL

Registering and Unregistering the DataContainer DLL
This section describes how to manually register and unregister the generated DataContainer DLL.
To register the DataContainer DLL:

1. Copy the generated DataContainer DLL to <IG-XL Install>\bin.
2. Open the Microsoft .Net Framework SDK command prompt: Start>All Programs>Microsoft .NET Framework SDK>SDK Command Prompt.
3. Execute this command in the command prompt:

regasm /tlb <FileName.dll>
Enter the DLL name along with the full path.
This command generates the required .tlb file from the given assembly name and then registers it.
The DataContainer can now be loaded by IG-XL.

☒ **Note:**
If the DataContainer DLL is copied to any local path other than <IG-XL Install>\bin, then you must copy all referenced DLLs to the same local path to register the DataContainer DLL successfully.
To unregister the DataContainer DLL:

1. Open the Microsoft .Net Framework SDK command prompt: Start>All Programs>Microsoft .NET Framework SDK>SDK Command Prompt.

2. [Execute this command in the command prompt:](#)

[regasm /unregister <FileName.dll>](#)
[Enter the DLL name along with the full path.](#)

< >		
	2015 Teradyne, Inc. All rights reserved.	

dISDKUsing.2.28



Using Datalog SDK : Using DataConsumer and DataContainer in IG-XL : Loading a Custom DataConsumer

Loading a Custom DataConsumer

During datalogging in the test program, perform the following steps to enable the custom consumer so that IG-XL loads the custom dataconsumer DLL:

```
Public sub CustomConsumerTest ()
```

```
    TheExec.Datalog.Setup.DatalogSetup.DatalogOn = True
```

```
    TheExec.Datalog.Setup.DataStreams("StreamName").Outputs.Custom.Create _  
        "D:\CustomDataConsumer.dll", "DataConsumer.CustomDataConsumer"
```

```
    TheExec.Datalog.Setup.DataStreams("StreamName").Outputs.Custom.Enable
```

```
    TheExec.Datalog.ApplySetup
```

```
End Sub
```

Note these points:

- You can enable custom data consumers for any stream and use them like other Teradyne consumers (STDF, Text, and DataFilter).
- In the above method, custom output is enabled for the default datalog stream.
- Provide the DLL name along with the path. For example, your DLL is D:\CustomDataConsumer.dll. Specify the class name with the namespace. Therefore, if the namespace name is DataConsumer and the class name is CustomDataConsumer, then give the class name as DataConsumer.CustomDataConsumer.

- If the DLL is present in the network path, then it requires that the network path be mapped to the tester’s machine for the consumer to be loaded.
-
- You can create and enable a custom consumer in the OnProgramLoaded() interpose function if the Events are enabled and if system events are required to be logged from the Program Load event.

		
	2015 Teradyne, Inc. All rights reserved.	

dISDKUsing.2.29



Using Datalog SDK : Using DataConsumer and DataContainer in IG-XL : Sending Custom Data in DataContainer

Sending Custom Data in DataContainer

The custom consumer is now enabled, and data can be sent through the container to the custom consumer. To send data through the container from IG-XL, link the generated data container DLL either statically or dynamically from VBA.

Dynamic Linking

Dynamic linking can be done by creating an object for the DataContainer class using the CreateObject method. Use the code below to create an object of the data container class and to send an integer value and an integer array to the container.

Public Function SendDataToContainer_DynamicLinking ()

```

Dim container As Object
Set container = CreateObject ("SampleDataContainer.SampleContainer")
If container Is Nothing Then
    Exit Function
End If
container.IntegerValue = 10
Dim intArr (2) As Long
intArr (0) = 1
intArr (1) = 2
intArr (2) = 3
container.IntegerArray = intArr
TheExec.Datalog.SendData container
End Function

```

Static Linking: Adding a DataContainer Reference

Static linking can be done in two ways:

- [Manually, you can do it by selecting](#) Tools>References from the VBA editor, selecting the generated Custom Data Container type library (*.tlb file), and clicking OK.

- [You can also execute the following line from VBA:](#)

[Application.ActiveWorkbook.VBProject.References.AddFromFile "Type Library name"](#)

[Execute this line from the OnProgramLoaded\(\) interpose function.](#)

[After linking, data container class members can be accessed from the VBA. Use the following sample code to send a string value and string array to the container:](#)

```
Public Sub SendDataToContainer()
```

```
Dim sample As SampleContainer
```

```
Set sample = New SampleContainer
```

```
If sample Is Nothing Then
```

```
Exit Sub
```

```
End If
```

```
sample.stringValue = "Sample DataContainer-StaticLinking"
```

```
    Dim strArr (3) As String
```

```
    strArr (0) = "String1"
```

```
    strArr (1) = "String2"
```

```
    strArr (2) = "String3"
```

```
    strArr (3) = "String4"
```

```
    sample.SetStringArray strArr
```

```
    TheExec.DataLog.SendData sample
```

```
    Set sample = Nothing
```

```
End Sub
```

[When a string value and string array is passed to the container, the DataContainerPacket can be handled as shown below in the custom data consumer project.](#)

```
protected virtual void HandleDataContainerPacket(DataPacket inPacket)
```

```
{
```

```
    try
```

```
    {
```

```
        DataContainerPacket container = (DataContainerPacket)inPacket;
```

```
        if (container != null)
```

```
        {
```

```
            string stringValue = container.StringValue;
```

```
            string[] stringArray = container.StringArray;
```

```
            private StreamWriter dataContainerFile;
```

```
if (dataContainerFile == null)
{
    dataContainerFile = new StreamWriter(m_eventsFileName, true);
    dataContainerFile.AutoFlush = true;
}
dataContainerFile.WriteLine ("String Value : " + stringValue);
if (stringArray!= null)
{
    dataContainerFile.Write("String Arrays : ");
    foreach (string value in stringArray)
    {
        dataContainerFile.Write (value + "," );
    }
    dataContainerFile.WriteLine();
}
}
}
}
catch (Exception exp)
{
    throw new CustomConsumerException ("SampleDataConsumer:DataContainer Packet handling
error " + exp.ToString());
}
}
```

<

>

dISDKUsing.2.30



Using Datalog SDK : Using DataConsumer and DataContainer in IG-XL : Receiving Data in a DataContainer

Receiving Data in a DataContainer

Validate and run the IG-XL test program to receive the data container data packet, which contains the custom data.

The following is sample VBT code for exec interpose functions that create and enable custom consumer and use data container.

Option Explicit

Function OnProgramLoaded()

On Error GoTo ErrHandler

TheExec.Datalog.Events.EnableAll

TheExec.Datalog.Setup.DatalogSetup.DatalogOn = True

Dim igxlinstalledpath As String

igxlinstalledpath = Environ("IGXLROOT")

Call TheExec.Datalog.Setup.DataStreams.Item.Outputs.Custom.Create(igxlinstalledpath +
"src\Datalog\Templates\SampleDataConsumer\bin\Release\DatalogSampleSDK.dll",
"SampleSDK.SampleDataConsumer.SampleDataConsumer")

Call TheExec.Datalog.Setup.DataStreams.Item.Outputs.Custom.Enable

TheExec.Datalog.ApplySetup

Dim container As Object

Set container = CreateObject("SampleSDK.SampleDataContainer.FileNamesContainer")

container.CustomFileName = "CustomLog_DatalogStream.txt"

```
container.EventsFileName = "EventsLog_DatalogStream.txt"
container.DataContainerFileName = "ContainerLog_DatalogStream.txt"
container.StreamID = "DatalogStream_customConsumer"
```

```
TheExec.Datalog.SendData container
```

```
Set container = Nothing
```

```
TheExec.Datalog.Setup.DataStreams.CreateStream \("Stream1"\)
```

```
' Need to do ApplySetup() after calling CreateStream(), then only the
' stream would be created. After that, we can access the stream using the
' 'item' property
```

```
TheExec.Datalog.ApplySetup
```

```
Call
```

```
TheExec.Datalog.Setup.DataStreams.Item("Stream1").Outputs.Custom.Create(igxlinstalledpath +
"\src\Datalog\Templates\SampleDataConsumer\bin\Release\DatalogSampleSDK.dll",
"SampleSDK.SampleDataConsumer.SampleDataConsumer")
```

```
Call TheExec.Datalog.Setup.DataStreams.Item("Stream1").Outputs.Custom.Enable
```

```
TheExec.Datalog.ApplySetup
```

```
Dim container2 As Object
```

```
Set container2 = CreateObject("SampleSDK.SampleDataContainer.FileNamesContainer")
```

```
container2.CustomFileName = "CustomLog_Stream1.txt"
```

```
container2.EventsFileName = "EventsLog_Stream1.txt"
```

```
container2.DataContainerFileName = "ContainerLog_Stream1.txt"
```

```
container2.StreamID = "Stream1_customConsumer"
```

```
TheExec.Datalog.SendData container2
```

```
Set container2 = Nothing
```

```
Exit Function
```

```
ErrorHandler:
```

```
LocalHandleExecIPErr "OnProgramLoaded"
```

```
End Function
```

		
	2015 Teradyne, Inc. All rights reserved.	

dISDKUsing.2.31

<	>
-----------------	-----------------

Using Datalog SDK : Debugging a Custom DataConsumer

Debugging a Custom DataConsumer

This section provides a procedure for debugging custom data consumer code. The following topics are included:

- [Build the Custom DataConsumer](#)
- [Attach to the DatalogServer Process](#)
- [Enable Breakpoints](#)
- [Start Debugging](#)

<	>
-----------------	-----------------

dISDKUsing.2.32

<

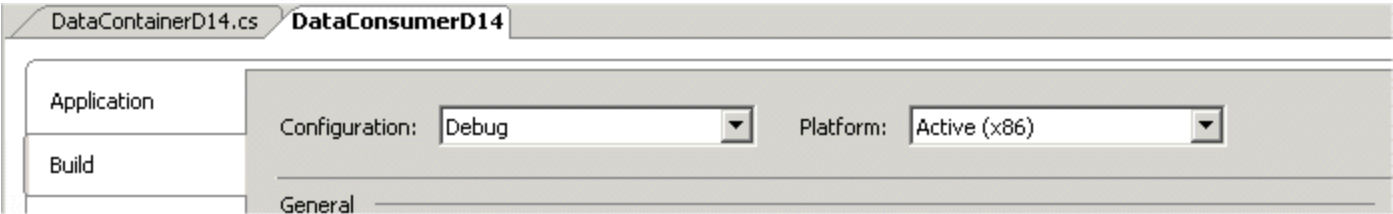
>

Using Datalog SDK : Debugging a Custom DataConsumer : Build the Custom DataConsumer

Build the Custom DataConsumer

Once the custom data consumer project is created, build the project with debug information. Follow these steps.

1. Right click the project name in the Solution Explorer and select Properties.
2. From the project Properties window, select the Build tab.



Build Tab in Solution Explorer

3. Select the Debug option for the Configuration property and then build the project.

<

>

dISDKUsing.2.33

<	>
-----------------	-----------------

Using Datalog SDK : Debugging a Custom DataConsumer : Attach to the DatalogServer Process

Attach to the DatalogServer Process

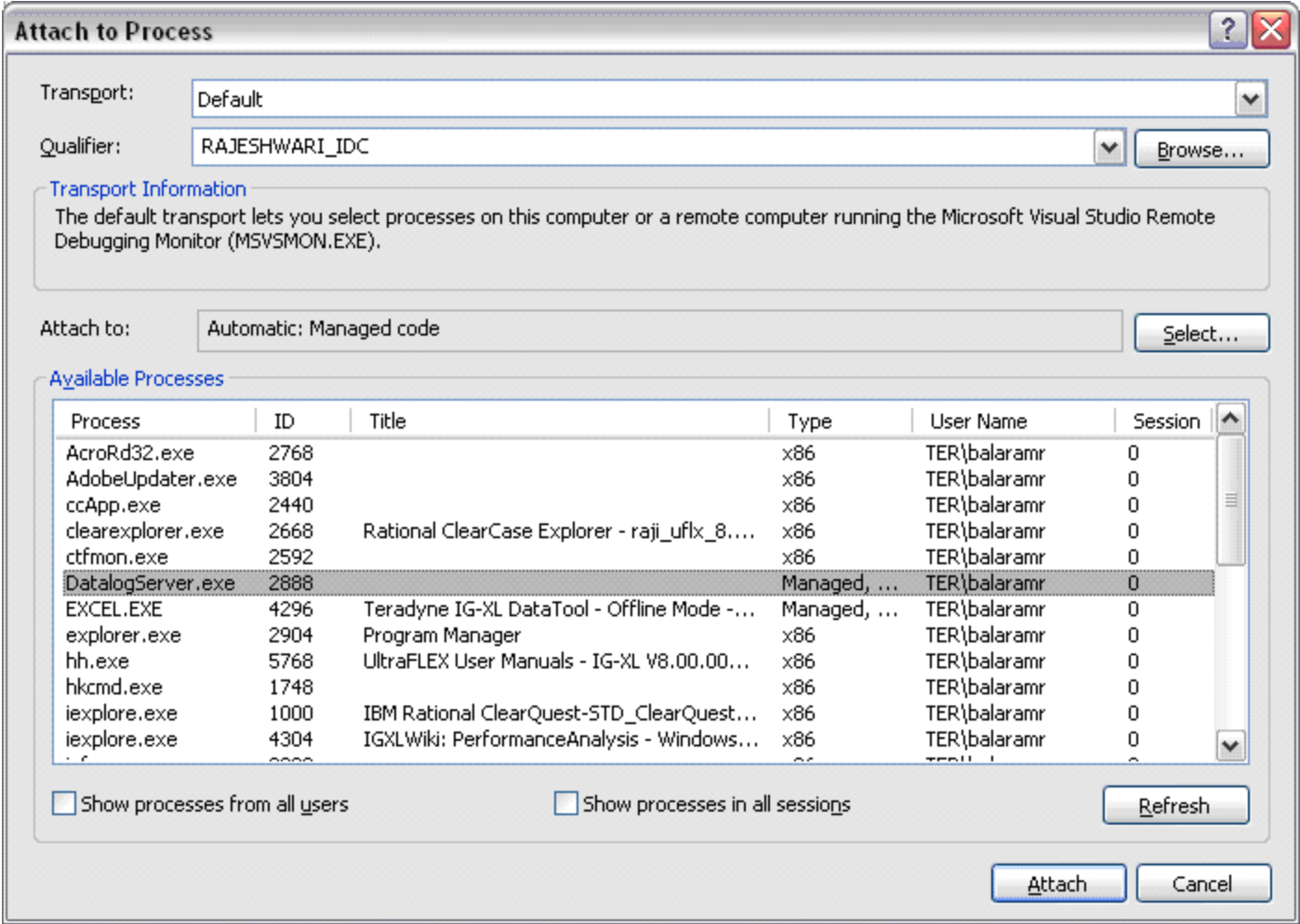
Once you have successfully built the custom data consumer project with the debug information, attach the custom data consumer project the datalog server process to enable debugging of the code.

To attach the project to the datalog sever process, follow these steps:

- | | |
|----|--|
| 1. | Launch the IG-XL test program that uses the custom data consumer that is built. |
| 2. | In the Visual Studio IDE where the custom data consumer project is loaded, select Debug>Attach to Process. |

The Attach to Process window appears as below.

- | | |
|----|--|
| 3. | From the list in the window, select DatalogServer.exe and click the Attach button. |
|----|--|



Attach to Process Window

<

>

2015 Teradyne, Inc. All rights reserved.

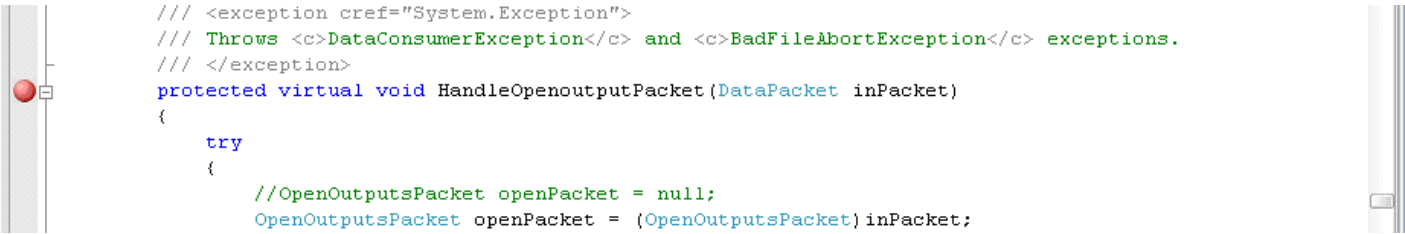
dISDKUsing.2.34



Using Datalog SDK : Debugging a Custom DataConsumer : Enable Breakpoints

Enable Breakpoints

After attaching the custom data consumer project to the Datalog Server process, place breakpoints in the methods that are to be debugged. For example, to debug the code for handling the OpenOuput packet, place a breakpoint in the HandleOpenoutputPacket method. To place a breakpoint over a particular line, place the cursor on that line and press the F9 function key, as shown in the following figure.



Placing a Breakpoint in a Method

The breakpoint is enabled when the custom dataconsumer DLL is loaded by the DatalogServer, which happens when the following set of statements is executed from VBA:

```
TheExec.Datalog.Setup.DatalogSetup.DatalogOn = True
TheExec.Datalog.Setup.DataStreams.Item.Outputs.Custom.Create "D:\CustomDataConsumer.dll",
"DataConsumer.CustomDataConsumer."
TheExec.Datalog.Setup.DataStreams.Item.Outputs.Custom.Create.Enable
TheExec.Datalog.ApplySetup
```



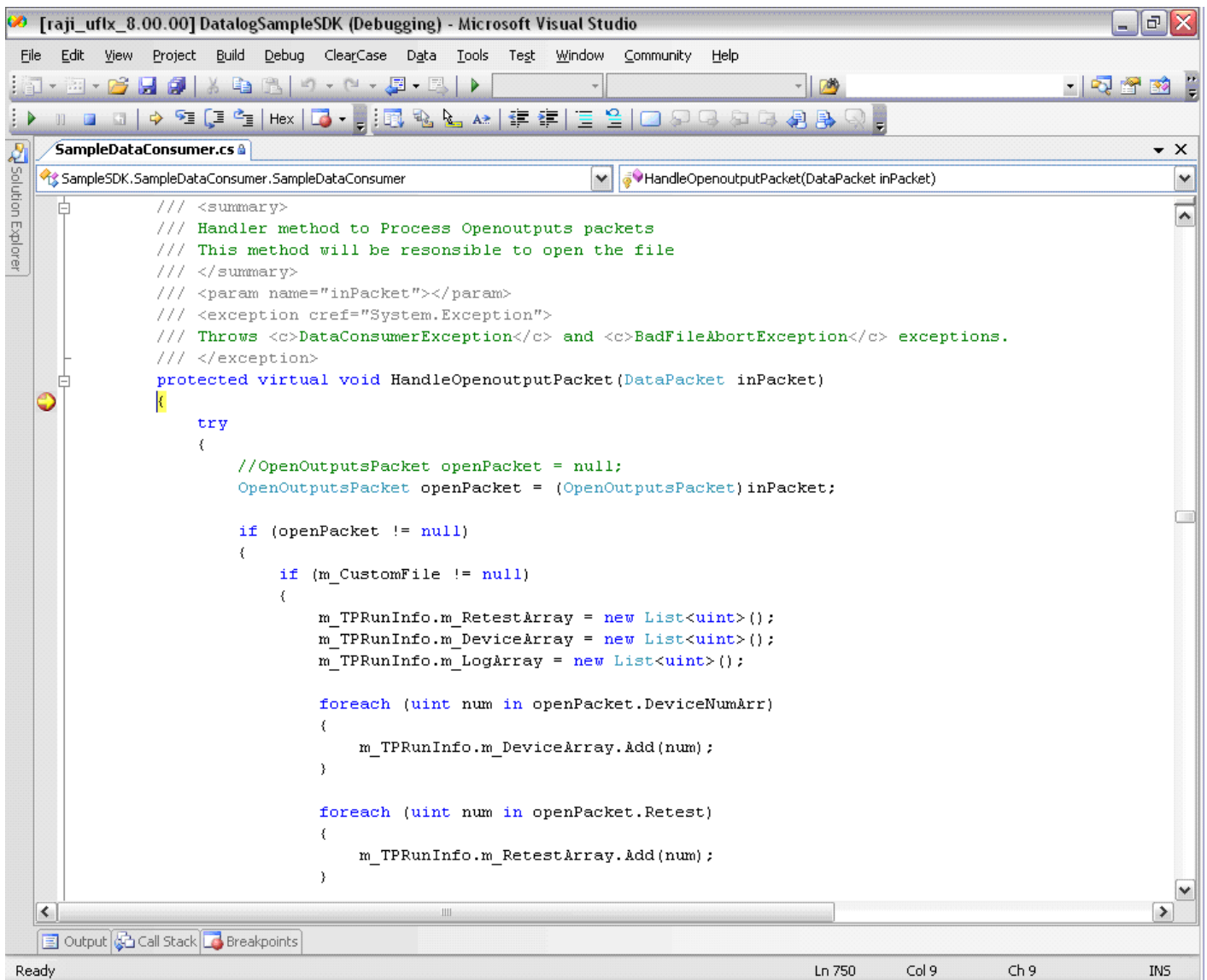
dlSDKUsing.2.35



Using Datalog SDK : Debugging a Custom DataConsumer : Start Debugging

Start Debugging

Once the breakpoint is enabled in the custom data consumer project, execute the test program. During execution, a breakpoint placed in a particular packet handler is hit when the custom data consumer receives the corresponding packet. For example, the breakpoint placed in the `HandleOpenOutputPacket` method is hit when the custom data consumer receives the `OpenOutputPacket`.



Hitting a Breakpoint

Once the breakpoint is hit in the packet handler method (with the yellow cursor), continue debugging using the F10 or F11 function keys.

Because the custom dataconsumer is processing asynchronously in a separate process, hitting a breakpoint in a custom dataconsumer does not pause IG-XL execution. Also, because every DataConsumer operates in its own thread, pausing a DataConsumer does not pause execution for any other DataConsumer.

Note these points:

- If you are attached to the custom dataconsumer and you kill the DatalogServer process, you must restart IG-XL.
- If the DatalogServer process hangs because of the custom dataconsumer or debugging efforts, check to see if the process is still running in the Task Manager after IG-XL has shut down. If it is still running, manually stop the process using the Task Manager.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

dISDKUsing.2.36

<	>
-----------------	-----------------

Using Datalog SDK : Debugging a DataContainer

Debugging a DataContainer

This section provides a procedure for debugging custom data container code. You can debug the data container in the IG-XL process or in the DatalogServer process. Both approaches are described in the following steps:

- Build the Custom DataContainer
- Debug the DataContainer in the IG-XL Process
- Debug the DataContainer in the DatalogServer Process

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dISDKUsing.2.37

<	>
-----------------	-----------------

Using Datalog SDK : Debugging a DataContainer : Build the Custom DataContainer

Build the Custom DataContainer

Once the custom data container project is created, build the project with debug information. Follow these steps:

1.

Right click the project name in the Solution Explorer and select Properties.
2.

From the project Properties window, select the first tab.
3.

Select the Debug option for the Configuration property and build the project.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

dISDKUsing.2.38

<	>
-----------------	-----------------

Using Datalog SDK : Debugging a DataContainer : Debug the DataContainer in the IG-XL Process

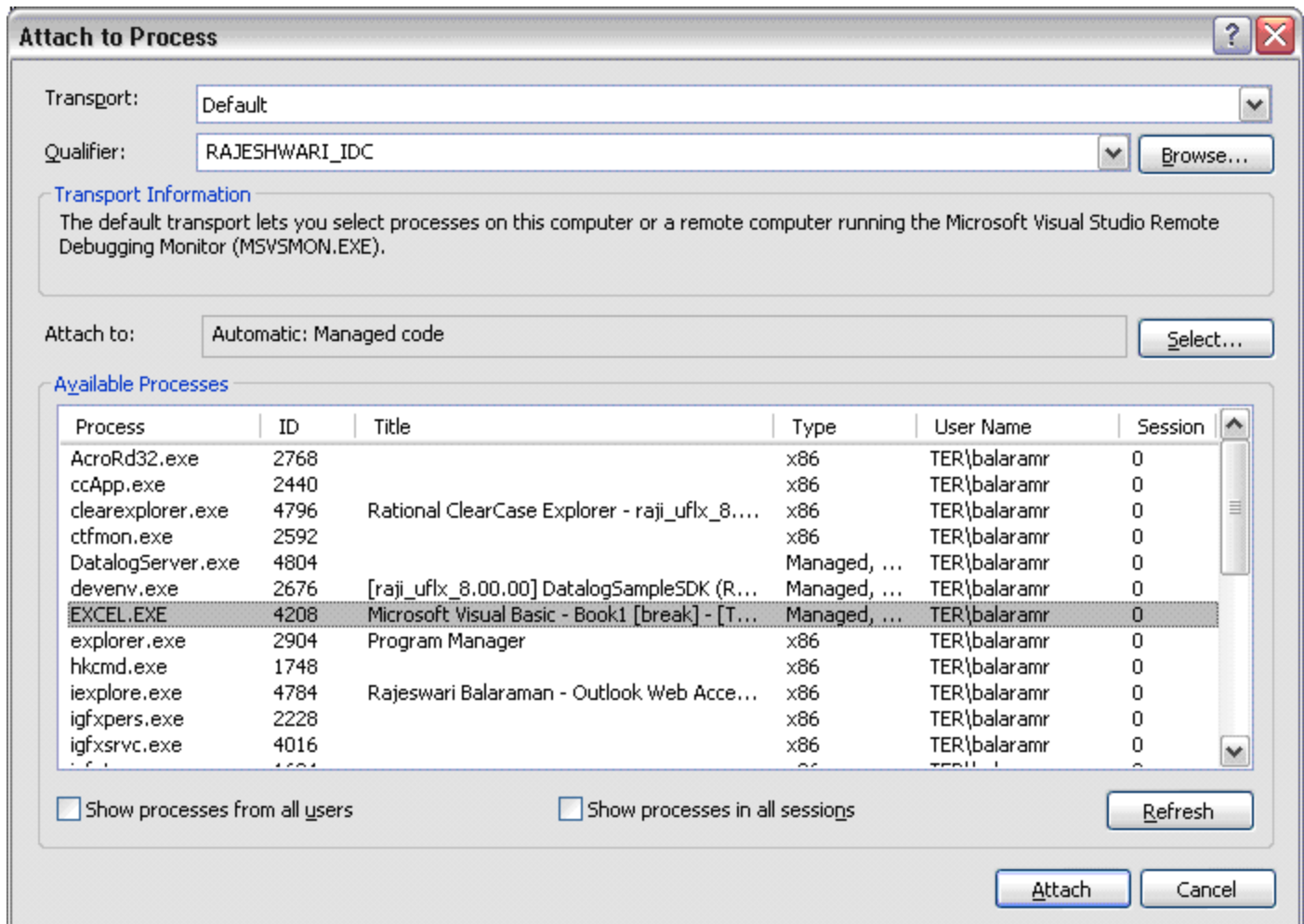
Debug the DataContainer in the IG-XL Process

This section describes how to debug the datacontainer code when the data is set in the container to be sent to the datalog. To send data to the datalog through the container, first IG-XL creates the data container object and then sets values to it. Thus, the datacontainer code must be debugged in the IG-XL process while the data is set in the container.

Attach to the IG-XL Process

Once the data container project is built successfully with the debug information, the data container project must be attached to the IG-XL process to enable debugging of the code. To attach the data container project to the IG-XL process, use the following steps:

- | | |
|----|--|
| 1. | Launch the IG-XL test program and link the generated data container DLL from the VBA either statically or dynamically. (See Sending Custom Data in DataContainer.) |
| 2. | In the Visual Studio IDE where the DataContainer project is loaded, select the Debug>Attach to Process. The Attach to Process window appears. |
| 3. | In the window, select the EXCEL.exe corresponding to the IG-XL test program and click the Attach button. |



Attach to Process Window

Enable Breakpoints

After the custom data container project is attached to the EXCEL process, place breakpoints in the methods that are to be debugged. For example, if the code for setting a double array in the data container project is to be debugged, then place a breakpoint in the Set property for the DoubleArray.

```

public double DoubleValue
{
    set
    {
        _doubleValue = value;
    }

    get
    {
        return _doubleValue;
    }
}

/// <summary>
/// Gets or sets the double array. The set property cannot be used when the

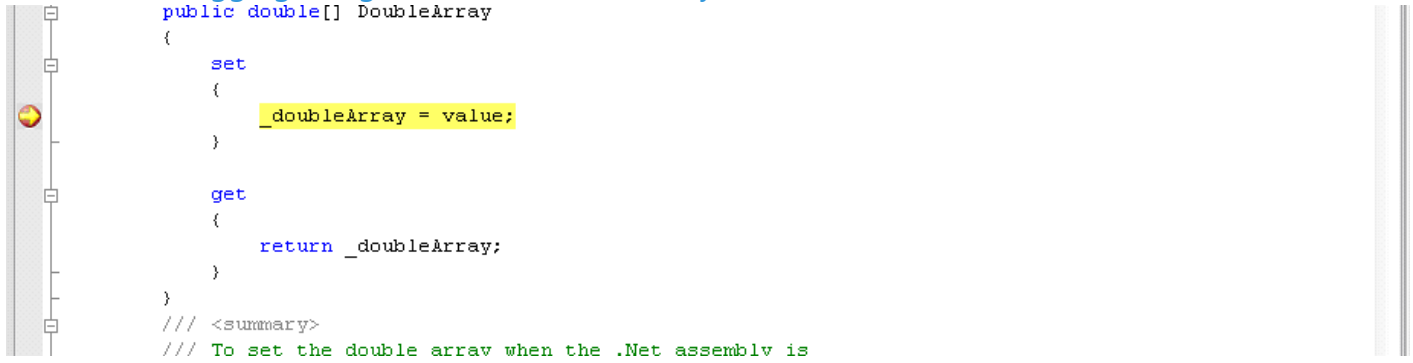
```

Placing a Breakpoint in a Method to Be Debugged

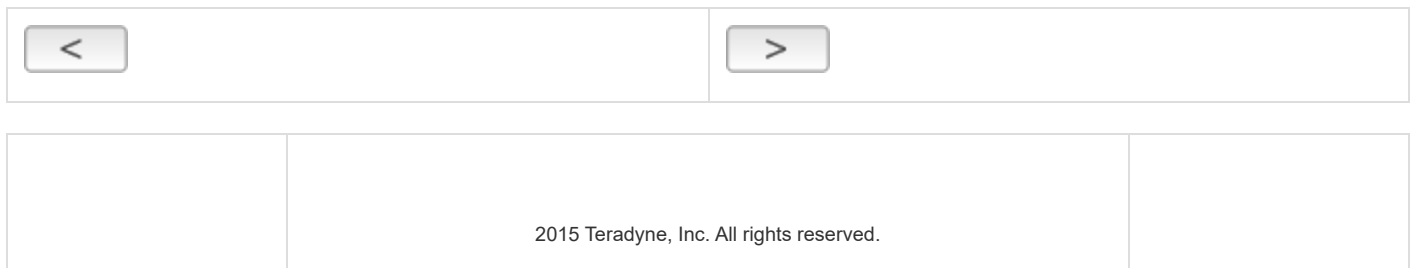
Start Debugging

Once you enable the breakpoint in the custom data container project, execute the test program. When the VBT language for setting the double array to the container is executed, the breakpoint

placed in the corresponding set property for the DoubleArray will be hit. Once the breakpoint is hit, continue debugging using F10 or F11 function keys.



Breakpoint Hit



dlsdkUsing.2.39



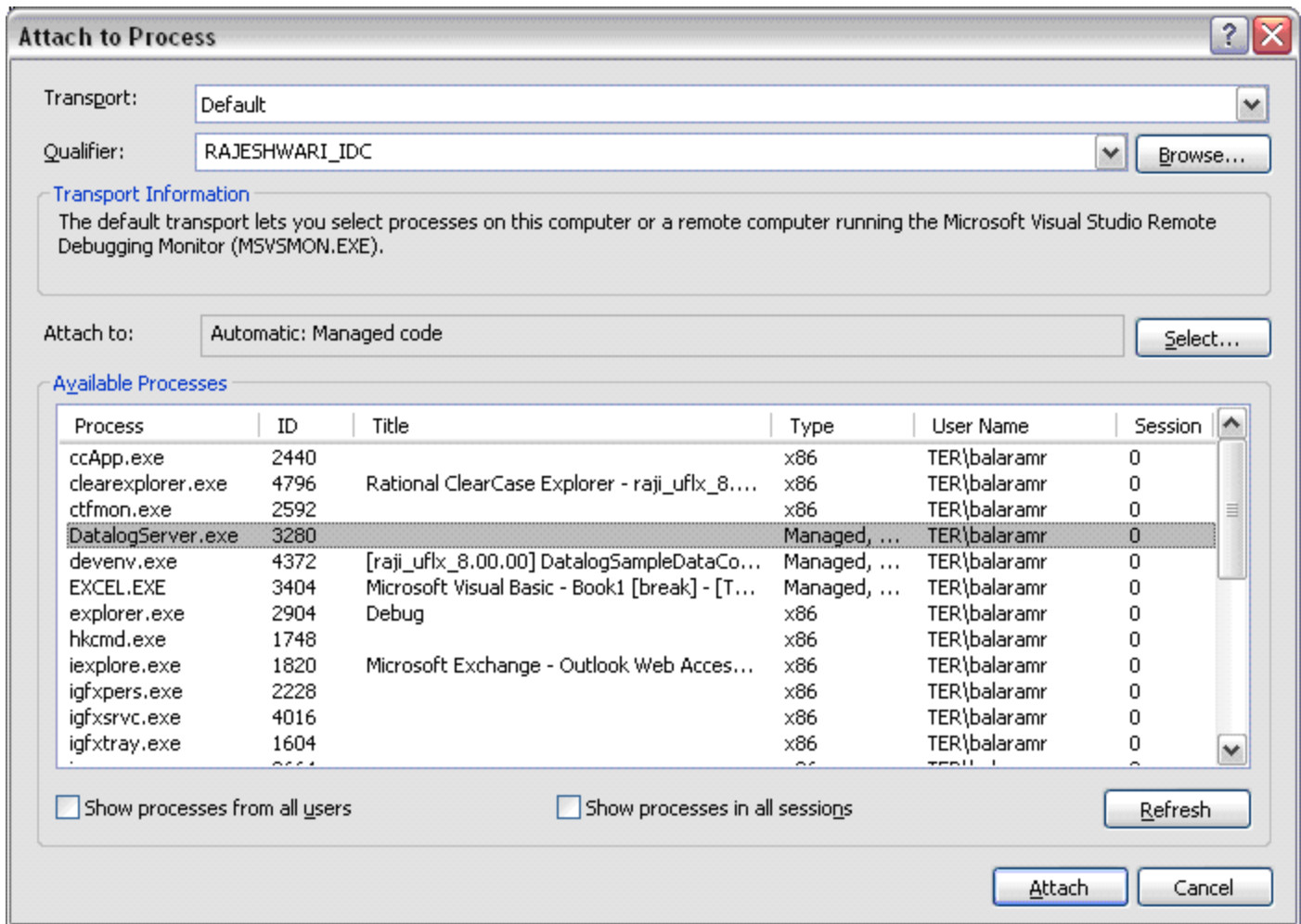
[Using Datalog SDK : Debugging a DataContainer](#) : Debug the DataContainer in the DatalogServer Process

[Debug the DataContainer in the DatalogServer Process](#)

This section describes how to debug the datacontainer code when the data is retrieved in the custom dataconsumer. After IG-XL sends the datacontainer object, the datalog server contains the object with all data being set. Hence, the datacontainer code must be debugged with the datalog server process when the consumer tries to get the data.

[Attach to the DatalogServer Process](#)

Open the DataContainer project in another Visual Studio IDE and then attach the project to the DatalogServer process.



Attaching a Project

Enable Breakpoint

Based on the type of data sent in the container, place breakpoints in the corresponding data type get properties in the datacontainer project. For example, if a double array is sent in the container to the custom data consumer, place the breakpoint in the get property for User double array in the datacontainer project.

```

    _doubleArray = value;
}

get
{
    return _doubleArray;
}

/// <summary>
/// To set the double array when the .Net assembly is

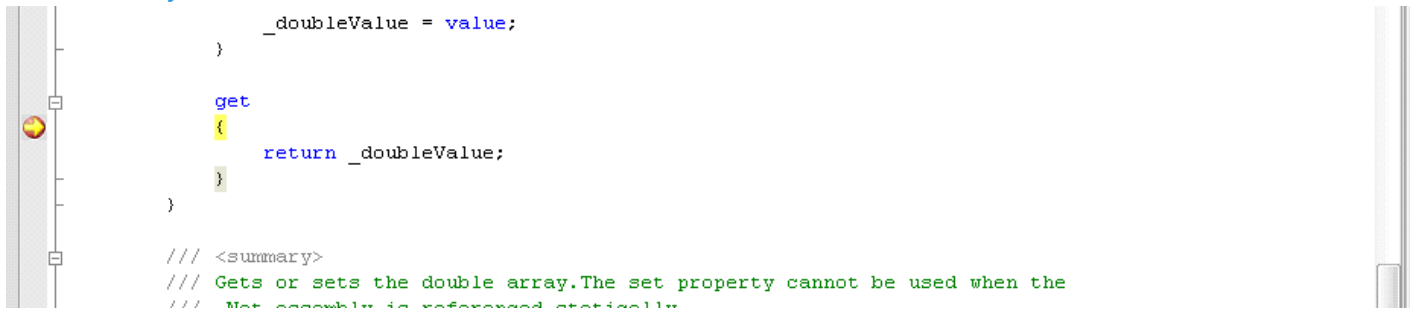
```

Placing a Breakpoint

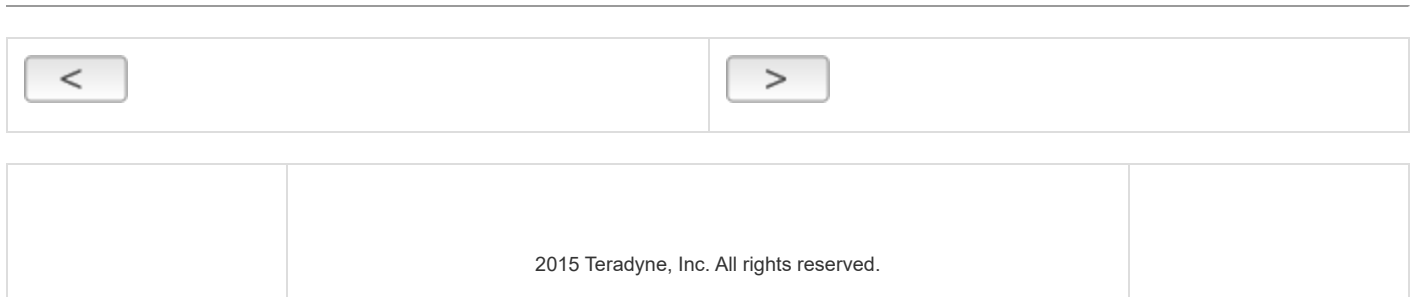
Start Debugging

During program execution, once the custom dataconsumer receives the datacontainer packet, the handler for the Datacontainer packet in the custom dataconsumer project is executed. In the handler method, when the consumer tries to get a particular piece of data that was sent in the container, then the breakpoint placed in the corresponding get property is hit. For example, if the

consumer tries to get the double array values, then the breakpoint in the get property for the DoubleArray would be hit.



Hitting a Breakpoint



dISDKUsing.2.40



Using Datalog SDK : FAQs

FAQs

- Why do I get a compilation error when I am trying to access an array of the datacontainer if the datacontainer is added as a reference to VB?

If the assembly is statically linked with VB (Adding a reference to the SampleDataContainer.tlb file through VBA Tools>References), the following compiler error is returned while trying to access the array:

"Function or interface marked as restricted, or the function uses an Automation type not supported in Visual Basic"

This is because VB version 6.0 cannot use the .NET method with the ParamArray parameter in Microsoft Visual Studio .NET. You can work around this problem by adding a separate method with a ref parameter that assigns the property to the private member. Refer to the following link: <http://support.microsoft.com/kb/327084>

Here is sample code for the method with the ref parameter.

```
public void SetIntegerArray(ref int[] intArray)
{
    _intArray = intArray;
}
```

- How do I use a dataconsumer assembly that is built from a different machine?

If the CustomDataConsumer is created in computer A, and you need to use the DataConsumer in computer B, copy the custom data consumer DLL generated from computer A to any path in computer B. The custom data consumer can now be accessed from computer B.

A custom data consumer created in computer A can be directly loaded in computer B if both the machines are mapped in a network.

- How do I use a datacontainer assembly that is built from a different machine?

If a DataContainer is created in computer A, and you need to use the datacontainer in computer B, use one of the following options to access members of the datacontainer.

Option 1:

- Copy the generated datacontainer type library file (*.tlb file) from computer A to any path inside computer B. (Make sure the assembly is a strong named assembly.)
- Launch an IG-XL workbook in computer B.
- Open the Excel references dialog for that workbook and locate the copied tlb file. Add reference to this tlb file and click OK.

The member of the data container that was built in computer A can now be accessed from computer B.

Option 2:

- Copy the generated datacontainer DLL from computer A to <IG-XL Install>\bin folder in computer B.
- Register the DLL using the command `ateregasm /codebase dllname`. (Enter the Dll name along with the full path.) See Registering and Unregistering the DataContainer DLL.
- Launch an IG-XL workbook and create a test program in computer B.
- Write a VBT method to create an object for the DataContainer class using the CreateObject method.

Once the object is successfully created, all members of the DataContainer can be accessed.

- How can I generate a GUID when using VS2010?

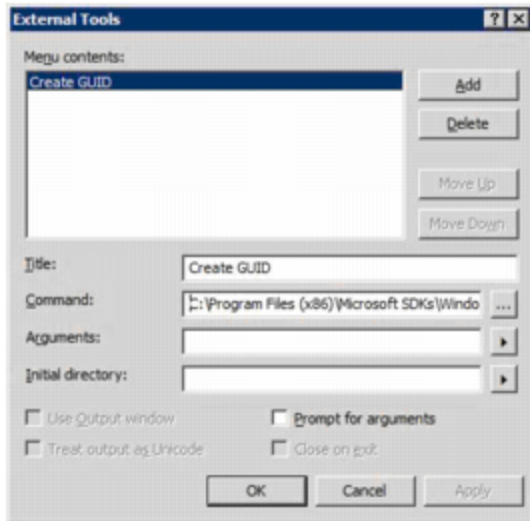
When you are using VS2010, use the following steps to generate a new GUID:

a. [Select](#) Tools> External Tools. The External Tools window appears.

b. In the External Tools window, enter the following and click OK:

Title: CreateGUID.

Command: C:\Program Files (x86)\Microsoft SDKs\Windows\v7.0A\bin\guidgen.exe

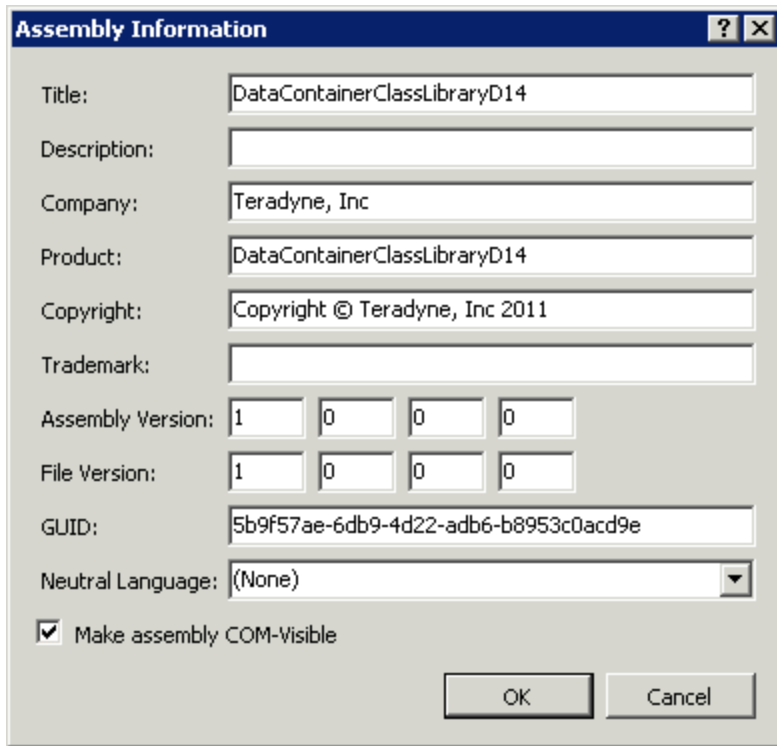


External Tools Window

The CreateGUID option will now be available in the Tools menu.

- [How do I modify the project name and copyright information?](#)

In the Application property settings, select the Assembly Information tab. You can modify the project name in the Title tab and the copyright information in the Copyright tab available in the Assembly Information window as shown below.



Assembly Information

Title: DataContainerClassLibraryD14

Description:

Company: Teradyne, Inc

Product: DataContainerClassLibraryD14

Copyright: Copyright © Teradyne, Inc 2011

Trademark:

Assembly Version: 1.0.0.0

File Version: 1.0.0.0

GUID: 5b9f57ae-6db9-4d22-adb6-b8953c0acd9e

Neutral Language: (None)

☒ Make assembly COM-Visible

OK Cancel

Assembly Information Window

- How do I install the custom data consumer template when using VS 2008 or VS 2010?

VS 2008: From the folder <IG-XL Install>\src\Datalog\Templates, copy the DataConsumerTemplate.zip to the folder My Documents\Visual Studio 2008\Templates\ProjectTemplates\Visual C#.

VS 2010: From the folder <IG-XL Install>\src\Datalog\Templates, copy the DataConsumerTemplate.zip to the folder My Documents\Visual Studio 2010\Templates\ProjectTemplates\Visual C#.

- How do I install the custom data container template when using VS 2008 or VS 2010?

VS 2008: From the folder <IG-XL Install>\src\Datalog\Templates, copy the DataContainerTemplate.zip to the folder My Documents\Visual Studio 2008\Templates\ItemTemplates\Visual C#.

VS 2010: From the folder <IG-XL Install>\src\Datalog\Templates, copy the DataContainerTemplate.zip to the folder My Documents\Visual Studio 2010\Templates\ItemTemplates\Visual C#.

<		>	
	2015 Teradyne, Inc. All rights reserved.		